

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ ПЛАНИРОВАНИЯ ЗАДАЧ С
УЧЁТОМ ИХ МЕСТОПОЛОЖЕНИЯ И ВРЕМЕННЫХ ПАРАМЕТРОВ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Студент: _____ / Тютючкин Семен Владимирович, 221–329 /
подпись *ФИО, группа*

Руководитель ВКР: _____ / Васильев Денис Борисович /
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Тема ВКР	Разработка веб-приложения для планирования задач с учётом их местоположения и временных параметров
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Автоматизированное формирование оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.
Основные функции	1. Управление задачами: создать, изменить, удалить 2. Оптимизация плана выполнения задач 3. Web-интерфейс
Используемые технологии и платформы	Golang, JavaScript, TypeScript, React, Docker, PostgreSQL, HTML, CSS

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Исследовать проблему оптимизации выполнения задач с гео-параметрами и временными параметрами 2. Проанализировать существующие продукты для оптимизации выполнения задач с гео-параметрами и временными параметрами 3. Разработать свое приложение на основе анализа существующих продуктов
Состав технической документации	1. Список документации
Состав графической части	1. Список графической части

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«___»_____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«___»_____2026, _____ / Васильев Денис Борисович /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«___»_____2026, _____ / Тютчикин Семен Владимирович, 221–329/
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: Разработка веб-приложения для автоматизированного планирования маршрутов с учетом временных окон.

Цель работы: разработка веб-приложения для автоматизированного формирования оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.

Задачи: анализ предметной области и существующих программных решений; исследование алгоритмов маршрутизации с временными окнами; разработка архитектуры программной системы; реализация серверной части и алгоритма оптимизации маршрута; разработка пользовательского интерфейса с визуализацией маршрута, импортом и экспортом задач; интеграция с внешними картографическими сервисами; тестирование и оценка производительности.

Объект исследования — процесс планирования последовательности выполнения задач в городской среде с учетом пространственных и временных параметров.

Предмет — алгоритмы маршрутизации и методы оптимизации порядка выполнения задач с временными окнами.

Работа содержит Введение, 3 главы, Заключение и 2 Приложения; включает 11 рисунков, 13 таблиц и библиографический список из 61 источника.

В Первой главе выполнены анализ предметной области, постановка задачи, обоснование выбора технологий и сравнительный анализ аналогов. Во Второй главе представлены модель данных, архитектура серверной части, алгоритм оптимизации маршрута, проект пользовательского интерфейса, техническая реализация и руководство по развертыванию. В Третьей главе раскрыты вопросы внедрения и опытной эксплуатации, аспекты отказоустойчивости, тестирование и результаты нагрузочного анализа производи-

тельности. В Заключении приведены выводы, оценка результатов, практическая значимость и перспективы развития системы.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	10
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ	12
1.1 Исходное состояние объекта и характеристика предметной области	12
1.2 Постановка проблемы и формирование цели разработки	12
1.3 Обоснование выбора технологий и платформ	14
1.4 Анализ алгоритмических подходов к задаче маршрутизации с временными окнами	16
1.5 Обзор аналогов и сравнительный анализ	19
1.6 Выводы	21
2 РЕАЛИЗАЦИЯ	22
2.1 Модель данных	22
2.2 Архитектура системы	25
2.3 Алгоритм оптимизации маршрута	27
2.4 Интерфейс	34
2.5 Техническая реализация	40
2.6 Руководство пользователя и администратора	42
2.7 Выводы	47
3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ	48
3.1 Анализ внедрения продукта	48
3.2 Аспекты отказоустойчивости, надежности и производительности	50
3.3 Тестирование	57
3.4 Внедрение и сопровождение программного продукта	61
3.5 Выводы	62
ЗАКЛЮЧЕНИЕ	63
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	66
ПРИЛОЖЕНИЕ А	74

ПРИЛОЖЕНИЕ Б	81
------------------------	----

ВВЕДЕНИЕ

Рост числа задач, привязанных одновременно к адресу и временному окну, делает ручное планирование маршрута трудоемким и подверженным ошибкам, что формирует потребность в инструментах автоматизированного планирования. Аналогичные тенденции наблюдаются и в коммерческой логистике последней мили [32], где увеличение числа адресов доставки и сужение временных окон обслуживания делают задачу построения оптимального маршрута экономически значимой.

Применение информационных технологий в управлении временем рассматривается как фактор повышения эффективности планирования личной и профессиональной деятельности [31].

Существующие цифровые решения представлены либо приложениями для управления задачами, ориентированными на учет перечня дел без учета пространственного фактора, либо навигационными сервисами, формирующими маршрут без учета длительности выполнения задач. В результате пользователи вынуждены комбинировать несколько приложений, вручную сопоставляя адреса, интервалы времени и порядок посещения точек. Подобный подход характеризуется высокой трудоемкостью и ростом вероятности ошибок при увеличении количества задач [1].

Целью выпускной квалификационной работы является разработка веб-приложения для автоматизированного формирования оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать предметную область и существующие программные решения;

- исследовать алгоритмические подходы к маршрутизации с временными окнами;
- разработать архитектуру программной системы;
- реализовать серверную часть и алгоритм оптимизации маршрута с поддержкой дополнительных ограничений;
- разработать пользовательский интерфейс с визуализацией маршрута на карте, импортом и экспортом задач;
- реализовать интеграцию с внешними картографическими сервисами;
- провести тестирование разработанного решения;
- оценить производительность и масштабируемость системы.

Практическая значимость работы — единый инструмент планирования маршрута с временными окнами, доступный частным пользователям и малому бизнесу без коммерческой подписки.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

1.1 Исходное состояние объекта и характеристика предметной области

При наличии у задач временных окон (например, «с 9:00 до 12:00») пользователь вынужден вручную сопоставлять время перемещения, длительность выполнения и интервалы доступности каждой точки [10, 42], а с ростом числа задач трудоемкость такого планирования и вероятность ошибок быстро увеличиваются [1].

Целевой аудиторией системы являются:

- частные пользователи, планирующие личные дела в городе;
- мастера и специалисты на выезде;
- курьеры малого бизнеса;
- индивидуальные предприниматели;
- диспетчеры, формирующие небольшие маршруты вручную.

Особенностью данной предметной области является необходимость одновременного учета:

- географического расположения объектов;
- временных ограничений;
- длительности выполнения задач;
- минимизации времени перемещения и ожидания.

1.2 Постановка проблемы и формирование цели разработки

Анализ исходного состояния показал, что основной проблемой является отсутствие доступного инструмента, объединяющего управление задачами и автоматическую маршрутизацию с учетом временных окон.

Ручной подход:

- требует значительных временных затрат;
- не гарантирует оптимальность маршрута;
- становится неэффективным при увеличении числа точек;
- не обеспечивает формализованного учета временных ограничений.

Целью разработки является создание веб-приложения, обеспечивающего автоматизированное формирование оптимального маршрута выполнения задач с учетом их географического положения, временных окон и длительности выполнения.

На основании цели сформирован следующий перечень функций:

- создание, редактирование, удаление и клонирование задач;
- импорт задач из файлов формата CSV и XLSX (Excel);
- экспорт задач в форматах CSV и XLSX (Excel).
- получение матрицы расстояний через внешние API;
- вычисление оптимальной последовательности выполнения задач;
- поддержка дополнительных ограничений маршрута: фиксация начальной и конечной точки, задание порядка выполнения пар задач;
- выбор способа передвижения (автомобиль, пешком, общественный транспорт);
- расчет времени прибытия, ожидания и завершения для каждой остановки;
- визуальное обнаружение конфликтов временных окон;
- визуализация маршрута на карте;

Для обеспечения соответствия назначению определены следующие нефункциональные требования:

- время ответа сервера — не более 2 секунд при типовой нагрузке 5 запросов в секунду;
- поддержка современных браузеров (Google Chrome версии 148 и выше, Yandex Browser версии 25 и выше);

- масштабируемость до 100 одновременных пользователей;
- устойчивость к повторным запросам внешних API за счет кэширования.

1.3 Обоснование выбора технологий и платформ

1.3.1 Выбор серверной платформы

В качестве языка для серверной части выбран Go [2] со следующими характеристиками, существенными для разрабатываемого приложения:

- статическая типизация, снижающая риск ошибок на этапе компиляции;
- встроенная модель конкурентности на основе горутин [11], обеспечивающая обслуживание большого числа одновременных HTTP-соединений без выделения отдельного потока операционной системы на каждый запрос;
- компиляция в нативный исполняемый файл, что дает малый объем образа контейнера и предсказуемое потребление ресурсов.

1.3.2 Выбор СУБД

В качестве системы управления базами данных выбрана PostgreSQL [3]. Обоснование:

- типизированные временные типы (TIMESTAMPTZ, DATE, TIME), удобные для хранения временных окон задач;
- непрерывное промышленное применение с 1996 года, стабильный релизный цикл и соответствие стандарту SQL:2016 [3];
- открытая лицензия и отсутствие лицензионных платежей при коммерческой эксплуатации.

1.3.3 Выбор клиентских технологий

Для реализации пользовательского интерфейса выбран язык TypeScript [21] с библиотекой React [4]. Причины выбора:

- компонентный подход, позволяющий повторно использовать элементы интерфейса (карточка задачи, форма, диалог);
- развитая экосистема готовых компонентов и картографических библиотек;
- статическая типизация TypeScript, повышающая качество и понятность кода по сравнению с JavaScript [23].

1.3.4 Выбор внешнего картографического сервиса

Для геокодирования адресов и визуализации маршрута выбран сервис «Яндекс JavaScript API и HTTP Геокодер» [19] (далее — Яндекс JS API). В таблице 1 представлено сравнение доступных картографических решений. Таблица 1 – Сравнение картографических API

Критерий	Яндекс JS API	Google Maps JS API	OpenStreetMap
Геокодер на русском языке	Да, включая топонимику РФ	Да	Качество в РФ ниже коммерческих провайдеров; обновление зависит от сообщества
Бесплатный лимит (геокодер)	1 000 за пр./сутки	200 \$ кредит/мес.	Без ограничений
Работа в России	Без ограничений	Может быть недоступен	Без ограничений

На основании сравнения (таблица 1) для целевой аудитории (российские пользователи) выбран Яндекс JS API: это единственный из рассмотренных вариантов, который одновременно поддерживает российскую топонимику.

мику, предоставляет встроенный мультимаршрутизатор для построения матрицы расстояний и гарантированно доступен в российском сетевом сегменте.

1.3.5 Итоговый технологический стек

В таблице 2 приведен обоснованный итоговый технологический стек.
Таблица 2 – Итоговый технологический стек разрабатываемой системы

Уровень	Технология	Обоснование
Серверная часть	Go 1.23 [20]	Компилируемый язык, простая модель конкурентности
База данных	PostgreSQL 16 [3]	Реляционность, поддержка типизированных временных типов, зрелость платформы
Клиентская часть	React + TypeScript [21]	Компонентный подход, статическая типизация, развитая экосистема
Сборка клиентской части	Vite 6 [22]	Оптимизированная сборка для производственной среды
Картографический API	Яндекс JS API 2.1	Русскоязычный геокодер, встроенный мультимаршрутизатор
Контейнеризация	Docker Compose [5]	Воспроизводимость среды, автоматическое применение миграций

1.4 Анализ алгоритмических подходов к задаче маршрутизации с временными окнами

Решаемая задача относится к классу «задача коммивояжера с временными окнами» (Travelling Salesman Problem with Time Windows, TSPTW) — обобщению классической задачи коммивояжера [15], в котором каждой вершине маршрута сопоставлен допустимый интервал начала обслуживания. Пользователь задает перечень адресов с временными окнами и длительно-

стью обслуживания, а система должна построить порядок посещения, минимизирующий суммарное время маршрута с учетом ожиданий перед открытием окон. Прибытие в вершину раньше открытия окна влечет ожидание; прибытие после закрытия окна делает обслуживание невозможным. Поскольку при отсутствии временных ограничений TSPTW сводится к TSP, она наследует его NP-трудность [25] и в общем случае не имеет известного полиномиального точного алгоритма решения.

1.4.1 Формальная постановка задачи

Пусть задан граф $G = (V, E)$, где $V = \{v_0, v_1, \dots, v_n\}$ — множество узлов (задач пользователя), E — множество дуг с матрицей расстояний d_{ij} и матрицей времен перемещения t_{ij} .

Каждый узел v_i ($i \geq 1$) характеризуется:

- временным окном $[a_i, b_i]$ — допустимым интервалом начала обслуживания;
- длительностью обслуживания s_i (минуты).

Если маршрутный агент прибывает в узел v_i в момент $\tau_i < a_i$, он ожидает до момента a_i ; если $\tau_i > b_i - s_i$ — обслуживание начать невозможно (нарушение ограничения).

Цель: найти порядок посещения всех узлов, минимизирующий суммарное время маршрута $T = \sum_i t_{ij} + \sum_i w_i$, где $w_i = \max(0, a_i - \tau_i)$ — время ожидания в узле v_i , при соблюдении условий $\tau_i \leq b_i - s_i$ для всех i .

На рисунке 1 приведен пример графа задачи TSPTW с тремя узлами. Ребра графа подписаны временем перемещения d_{ij} (в минутах), получаемым из матрицы расстояний внешнего API. Алгоритм NNH-TW выбирает оптимальный порядок посещения: $v_0 \rightarrow v_2 \rightarrow v_1$ — посещение ближайшей допустимой точки на каждом шаге.

Граф задачи маршрутизации с временными окнами (VRPTW)
 $G = (V, E)$, $V = \{v_0, v_1, v_2\}$, d_{ij} — время перемещения (мин), $[a_i, b_i]$ — временное окно

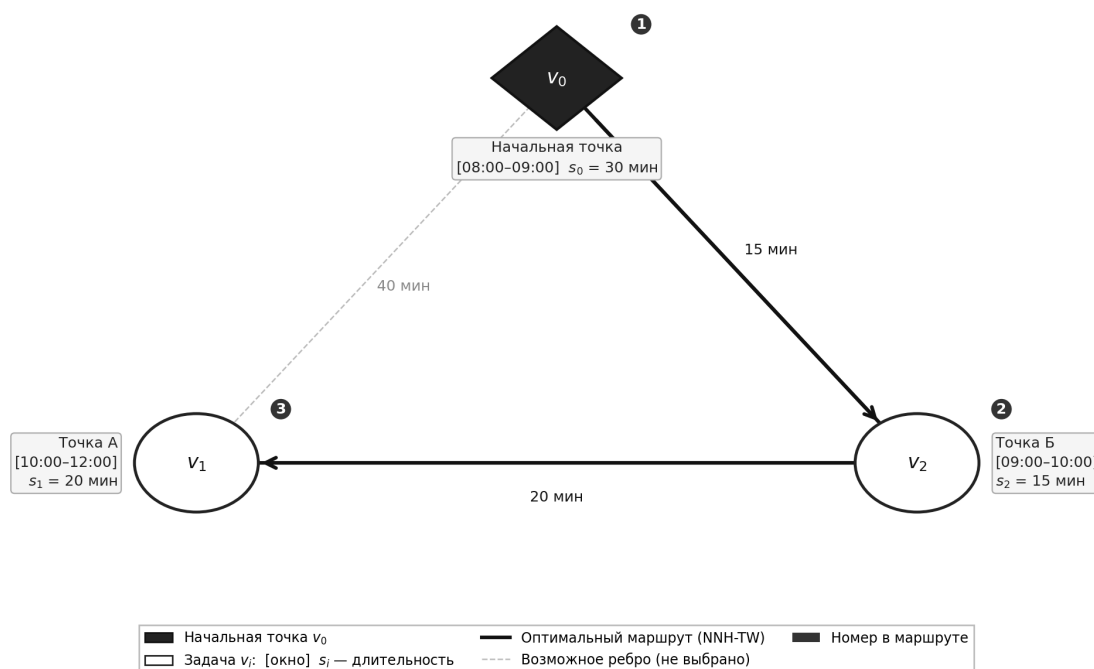


Рисунок 1 – Пример графа задачи коммивояжёра с временными окнами (TSPTW): d_{ij} — время перемещения между узлами (мин), $[a_i, b_i]$ — временное окно, s_i — длительность обслуживания; жирными стрелками выделен оптимальный маршрут $v_0 \rightarrow v_2 \rightarrow v_1$

1.4.2 Сравнение алгоритмических подходов

В таблице 3 приведен сравнительный анализ основных подходов к решению TSPTW применительно к задачам данной работы.

Таблица 3 – Сравнение алгоритмических подходов к задаче TSPTW

Подход	Сложность	Оптимальность маршрута
Полный перебор [25]	$O(n!)$	Да
Динамическое программирование (алг. Хелда–Карпа [15])	$O(2^n \cdot n^2)$	Да
Жадная эвристика NNH-TW (ближайший сосед с временными окнами и просмотром вперед)	$O(n^3)$	Нет
Муравьиный алгоритм [9, 33]	$O(iter \cdot n^2 \cdot m)$	Нет
Генетические алгоритмы [16]	$O(gen \cdot pop \cdot n)$	Нет

Здесь n — количество узлов маршрута, m — количество муравьев (агентов), $iter$ — число итераций, gen — число поколений, pop — размер популяции.

1.4.3 Обоснование выбора алгоритма

Для разрабатываемого веб-приложения, ориентированного на частных пользователей и малый бизнес, был выбран алгоритм «ближайшего соседа с временными окнами» (NNH-TW, Nearest Neighbour Heuristic with Time Windows) по следующим причинам:

- **Полиномиальная сложность** $O(n^3)$: на каждом из n шагов основного цикла для каждого из n кандидатов выполняется одношаговый просмотр вперед по n оставшимся узлам. Экспериментально (см. таблицу 11 в разделе «Производительность алгоритма оптимизации») среднее время работы при $n = 30$ составляет несколько микросекунд — на три-четыре порядка меньше задержки получения матрицы расстояний от внешнего API;

- **Предсказуемость**: время работы зависит от числа задач, что гарантирует выполнение нефункционального требования по времени ответа;

- **Качество решений**: жадные конструктивные эвристики обеспечивают приемлемое качество решений при низких вычислительных затратах [41], что достаточно для целей планирования личных задач и курьерских маршрутов малого масштаба.

1.5 Обзор аналогов и сравнительный анализ

Существующие программные решения, частично пересекающиеся с задачами разрабатываемой системы, можно разделить на три класса.

Первый класс — приложения для управления задачами (Todoist [35], Microsoft To Do [36]): реализуют полноценное создание, редактирование и удаление задач, поддерживают напоминания и мно-

гоплатформенную синхронизацию, однако полностью лишены функций геокодирования адресов и построения маршрутов.

Второй класс — навигационные сервисы (Google Maps [37], Yandex карты [44], 2GIS [45]): обеспечивают прокладку маршрутов по нескольким точкам, однако не поддерживают автоматическую оптимизацию порядка посещения и не предоставляют средств управления задачами [38].

Третий класс — специализированные логистические платформы (Routific [39], Relog [6], Яндекс Маршрутизация [7], Spoke [8]): сочетают геокодирование, оптимизацию маршрутов и учет временных окон, однако ориентированы на корпоративное управление автопарком (массовая обработка заказов, несколько водителей, диспетчеры) и недоступны для индивидуального применения без коммерческой подписки.

В таблице 4 приведен сравнительный анализ представителей всех трех классов.

Таблица 4 – Сравнительный анализ аналогов

Критерий	Todoist	Microsoft To Do	Google Maps	Routific	Яндекс Маршрутизация	Проектируемое решение
Управление задачами	Да	Да	Нет	Нет	Нет	Да
Геокодирование адресов	Нет	Нет	Да	Да	Да	Да
Автоматическая оптимизация маршрута	Нет	Нет	Нет	Да	Да	Да
Учет временных окон	Нет	Нет	Нет	Да	Нет	Да
Целевой сегмент	Частные пользователи	Частные пользователи	Массовый	Корпоративный	Средний и крупный бизнес	Частный пользователь и малый бизнес
Импорт/экспорт задач (CSV, XLSX)	Нет	Нет	Нет	Да	Да	Да
Модель распространения	Условно-бесплатная	Бесплатный	Бесплатный	Платная с триалом	Платная подписка	Бесплатный

Сравнение по таблице 4 показывает, что ни один из рассмотренных аналогов не обеспечивает одновременно управление задачами, автоматическую оптимизацию маршрута с учетом временных окон и доступность для частного пользователя без коммерческой подписки. Это подтверждает целесообразность разработки специализированного решения для целевой аудитории.

1.6 Выводы

В первой главе проведен анализ предметной области, выявлены ограничения существующих инструментов и обоснована актуальность разработки системы планирования маршрутов с временными окнами. Сформулированы цель и назначение продукта, определен перечень функциональных и нефункциональных требований.

Сравнительный анализ алгоритмических подходов к задаче TSPTW (полный перебор, динамическое программирование, жадные эвристики, метаэвристики) показал, что для целевого масштаба системы ($n < 50$) и требования по времени ответа наиболее подходящим является алгоритм ближайшего соседа с временными окнами (NNH-TW) с полиномиальной сложностью $O(n^3)$.

На основании сравнения технологий выбран стек разработки: Go, PostgreSQL, React/TypeScript, Vite, Яндекс JS API, Docker Compose. Анализ аналогов подтвердил целесообразность создания решения, ориентированного на частных пользователей и малый бизнес.

2 РЕАЛИЗАЦИЯ

2.1 Модель данных

2.1.1 Общая характеристика структуры хранения данных

На рисунке 2 представлена логическая модель данных разрабатываемого веб-приложения.

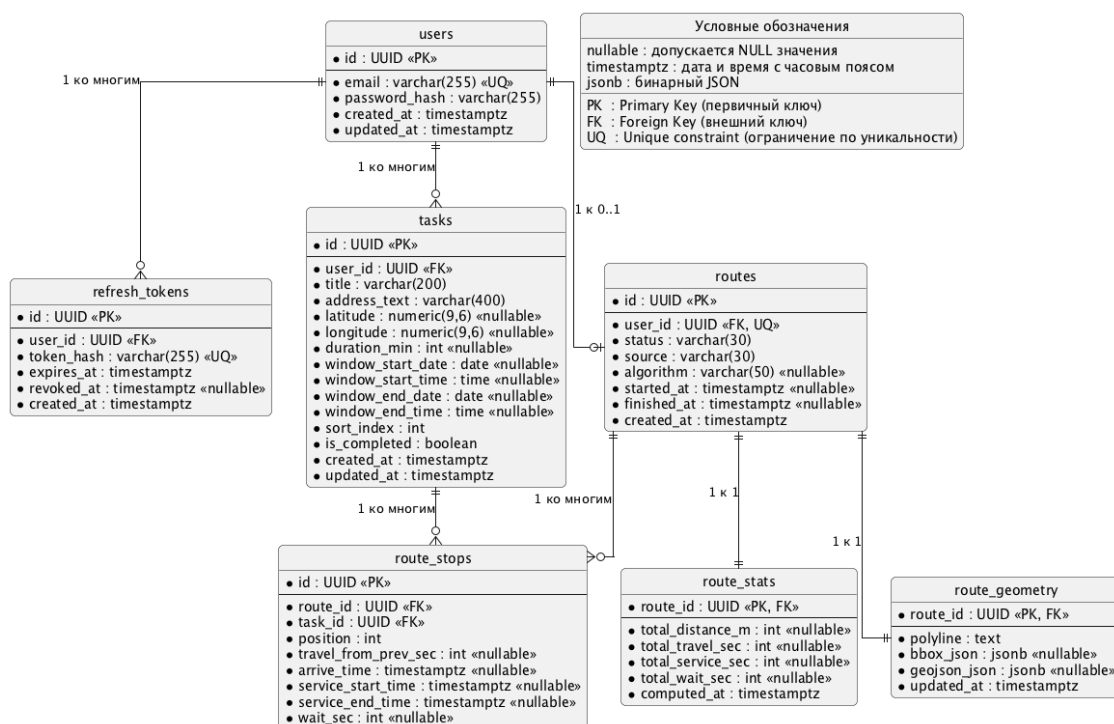


Рисунок 2 – Модель данных

Структура базы данных включает следующие функциональные группы сущностей:

- хранение информации о пользователях;
- хранение задач с географической привязкой;
- хранение маршрутов;
- хранение результатов вычислений.

2.1.2 Сущности, связанные с пользователями

Сущность `users` предназначена для хранения учетных записей пользователей. Первичный ключ `id` используется для установления связей со всеми зависимыми таблицами. Поле `email` выполняет роль логина и имеет ограничение уникальности, что исключает дублирование учетных записей. Пароль хранится в виде хэшированного значения, что обеспечивает безопасность аутентификации.

Сущность `refresh_tokens` используется для поддержки механизма продления авторизации. Каждый токен связан с конкретным пользователем через внешний ключ `user_id`. Хранение токена в виде хэша предотвращает его компрометацию в случае несанкционированного доступа к базе данных.

2.1.3 Сущность задач

Сущность `tasks` отражает основную предметную сущность системы — задачу, подлежащую выполнению.

Каждая запись содержит:

- связь с владельцем задачи (`user_id`);
- текстовое описание адреса (`address_text`);
- географические координаты (`latitude`, `longitude`), допускающие NULL для задач без адреса;
- длительность выполнения (`duration_min`), допускающую NULL для мгновенных задач;
- отдельные дату и время начала (`window_start_date`, `window_start_time`) и окончания (`window_end_date`, `window_end_time`) временного окна, что позволяет задавать ограничения как по дате, так и по времени;

- индекс сортировки (`sort_index`) для отображения порядка в пользовательском интерфейсе;
- признак выполнения (`is_completed`) для отметки завершенных задач.

2.1.4 Сущности маршрутов

Сущность `routes` описывает факт построения маршрута для конкретного пользователя. На каждого пользователя хранится не более одного маршрута, что обеспечивается ограничением уникальности `UNIQUE (user_id)`: при повторном запуске оптимизации предыдущая запись и связанные с ней остановки удаляются каскадно. Запись содержит сведения о статусе вычисления, источнике формирования последовательности, использованном алгоритме и времени создания.

Структура маршрута детализируется в сущности `route_stops`, где каждая запись соответствует отдельной точке маршрута. Поле `position` определяет порядок следования, а временные поля (`arrive_time`, `service_start_time`, `service_end_time`) позволяют зафиксировать результат работы алгоритма с учетом временных окон.

2.1.5 Хранение агрегированных показателей

Агрегированные характеристики маршрута вынесены в отдельную сущность `route_stats`. К таким показателям относятся суммарная длина маршрута, время перемещения, время выполнения задач и суммарное ожидание.

Выделение агрегированных данных в отдельную таблицу позволяет:

- уменьшить нагрузку на основную таблицу маршрутов;
- обновлять статистику независимо от структуры маршрута;
- отделить расчетные показатели от исходных данных.

2.1.6 Геометрия маршрута

Сущность `route_geometry` предназначена для хранения пространственного представления маршрута, используемого при визуализации на карте. Поле `polyline` либо `geojson_json` содержит геометрическое описание линии маршрута, полученное из внешнего API.

Разделение логической структуры маршрута и его геометрического представления позволяет обновлять визуальные данные без изменения структуры остановок.

2.2 Архитектура системы

2.2.1 Общие принципы

Серверная часть построена по принципам чистой архитектуры (Clean Architecture), предложенной Р. Мартином [14]. Зависимости направлены от внешних слоев (HTTP-обработчики, репозитории) к внутренним (доменные сущности и варианты использования), но не наоборот. Такое разделение делает бизнес-логику независимой от конкретного HTTP-фреймворка, СУБД и сторонних сервисов.

2.2.2 Слои архитектуры

Приложение организовано в четыре слоя, представленных в таблице 5.

Таблица 5 – Слои чистой архитектуры серверной части

Слой	Пакет	Назначение
Доменные сущности	<code>internal/domain/*/</code>	Структуры данных (Task, Route, User); без внешних зависимостей
Интерфейсы (порты)	<code>internal/domain/*/gate/</code>	Интерфейсы репозитория и оптимизатора; определяют контракт без реализации
Варианты использования	<code>internal/domain/*/story/</code>	Бизнес-логика: создание задач, пакетный импорт (task import), запуск оптимизации, управление сессией
Адаптеры	<code>internal/api/http/</code> и <code>internal/platform/postgres/</code>	HTTP-обработчики и SQL-репозитории; зависят от внешних библиотек

2.2.3 Паттерн замены алгоритма

В пакете `internal/domain/route/gate/` определен интерфейс `Optimizer` с единственным методом `Optimize(ctx, graph, startTimeUnix, constraints)`: на вход — граф задач, время выезда (секунды Unix) и структура дополнительных ограничений (фиксированные начальная/конечная точки и попарные ограничения порядка), на выходе — порядок обхода, тайминги остановок и агрегированная статистика. Конкретная реализация `NearestNeighborTW` регистрируется в модуле сборки зависимостей `internal/app/app.go`, поэтому замена алгоритма (например, на муравьиный или генетический) сводится к подключению новой реализации интерфейса без правки HTTP-обработчиков, репозитория и доменных сущностей.

2.2.4 Кэширование внешних запросов

Результаты обращений к внешнему сервису маршрутизации OSRM [17, 18] кэшируются на уровне приложения в оперативной памяти через обертку `CachedProvider` над интерфейсом `distance.Provider`. В качестве хранилища используется библиотека `hashicorp/golang-lru/v2` [40] (пакет `expirable`) — потокобезопасный LRU-кэш с ограничением размера и временем жизни записи (TTL). Стратегия LRU выбрана как одна из наиболее эффективных и наименее ресурсоемких политик вытеснения для кэшей с ограниченным объемом [30]. Ключом записи служит пара географических координат, округленных до шестого знака после запятой, что делает кэш инвариантным к порядку запросов одних и тех же адресов. Кэш снижает количество обращений к внешнему сервису, уменьшает задержки повторной оптимизации и повышает устойчивость системы при ограничениях со стороны API-провайдера.

2.3 Алгоритм оптимизации маршрута

2.3.1 Описание алгоритма

Для вычисления оптимального порядка посещения задач реализован алгоритм «ближайшего соседа с временными окнами» (Nearest Neighbour Heuristic with Time Windows, NNH-TW). Базовый алгоритм ближайшего соседа — одна из наиболее изучаемых конструктивных эвристик для задачи коммивояжера, традиционно используемая в качестве эталона при сравнении более сложных методов [24].

Алгоритм является жадной конструктивной эвристикой с одношаговым просмотром вперед (one-step look-ahead): на каждом шаге из допустимых кандидатов выбирается узел с наименьшим временем завершения обслуживания, при этом предпочтение отдается «безопасным» кандидатам —

тем, посещение которых не делает недостижимым ни один другой узел с временным окном. Внутри каждого класса (безопасные / рискованные) приоритет получает кандидат с более ранним дедлайном, что снижает вероятность пропуска срочных задач. Алгоритм также учитывает дополнительные ограничения: фиксацию начальной и конечной точки маршрута, а также попарные ограничения порядка посещения (предшествование).

2.3.2 Псевдокод алгоритма

Листинг 2.1 – Псевдокод алгоритма NNH-TW с просмотром вперед

```
1 Вход: граф  $G(V, E)$ ,  $startTimeUnix$ ,  $constraints\{startIdx,$   
    $\hookrightarrow endIdx, precedences\}$   
2 Выход: порядок обхода  $order[]$ , тайминги[], агрегированная  
    $\hookrightarrow$  статистика  
  
3 1.  $prereqs \leftarrow buildPrereqs(precedences)$   
4 2.  $endIdx \leftarrow constraints.endIdx$  (или -1)  
5 3. Если  $constraints.startIdx$  задан:  
6      $cur \leftarrow constraints.startIdx$   
7 Иначе:  
8      $cur \leftarrow$  узел с наиболее ранним окном среди допустимых  
9 4.  $visited[cur] \leftarrow true$ ;  $order \leftarrow [cur]$   
10 5.  $currentTimeSec \leftarrow startTimeUnix + duration[cur] * 60$   
  
11 6. Пока  $|order| < |V|$ :  
12    // Если остался только закреплённый конечный узел –  
    $\hookrightarrow$  ставим его  
13    а. Если  $endIdx \geq 0$  И все прочие посещены:  
14       Добавить  $endIdx$  в маршрут; выйти из цикла
```

```

15 // Проход 1: допустимый узел с минимальным
    ↪ completionTime + look-ahead
16 b. bestComp ← ∞; bestSafe ← false; bestDeadline ← ∞;
    ↪ next ← -1
17 c. Для каждого i ∈ visited, i ≠ endIdx:
18     Если не выполнены предшественники i: пропустить
19     arrivalSec ← currentTimeSec + travelSec[cur][i]
20     Если НЕ feasible(i, arrivalSec): пропустить
21     comp ← completionTime(i, arrivalSec)
22     safe ← НЕ causesWindowMiss(i, comp, G, visited,
    ↪ endIdx)
23     Если (safe > bestSafe) ИЛИ (safe = bestSafe И comp
    ↪ < bestComp)
24         ИЛИ (safe = bestSafe И comp = bestComp И
    ↪ deadline[i] < bestDeadline):
25         next ← i; bestComp ← comp; bestSafe ← safe

26 // Проход 2: резервный — минимальный completionTime без
    ↪ учёта окна
27 d. Если next = -1:
28     Для каждого i ∈ visited, i ≠ endIdx:
29         Если не выполнены предшественники i: пропустить
30         comp ← completionTime(i, arrivalSec)
31         Если comp < bestComp: next ← i

32 e. waitSec ← max(0, windowStart[next] - arrivalSec)
33 f. currentTimeSec ← arrivalSec + waitSec +
    ↪ duration[next] * 60
34 g. visited[next] ← true; order.append(next); cur ←
    ↪ next

35 7. Вернуть order, тайминги и статистику

36 feasible(i, arrivalSec):

```

```

37     Если windowEnd[i] < 0: вернуть true
38     Вернуть arrivalSec ≤ windowEnd[i] - duration[i] * 60

39 causesWindowMiss(cand, afterCandSec, G, visited, endIdx):
40     Для каждого j ∈ visited, j ≠ cand, j ≠ endIdx:
41         Если windowEnd[j] < 0: пропустить
42         Если НЕ feasible(j, afterCandSec +
            ↪ travelSec[cand][j]):
43             Вернуть true
44     Вернуть false

```

2.3.3 Блок-схема алгоритма

Блок-схема алгоритма выполнена в соответствии с условными обозначениями ГОСТ 19.701-90 [26] и представлена на рисунке 3.

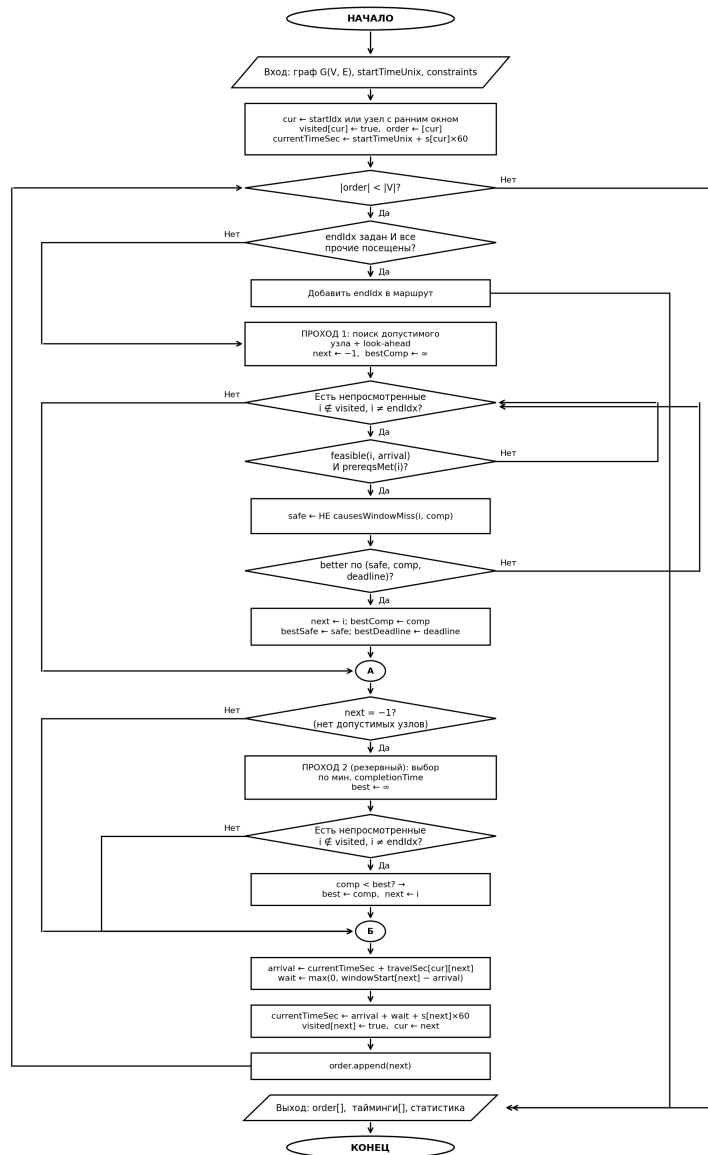


Рисунок 3 – Блок-схема алгоритма ближайшего соседа с временными окнами (NNH-TW)

2.3.4 Трассировка на числовом примере

Рассмотрим пример с тремя задачами для иллюстрации работы алгоритма. Исходные данные приведены в таблице 6, матрица времен перемещения — в таблице 7.

Таблица 6 – Исходные данные задач для трассировки

Узел	Адрес	Временное окно	Длительность
v_0	Офис (начало)	08:00–09:00	30 мин
v_1	Точка А	10:00–12:00	20 мин
v_2	Точка Б	09:00–10:00	15 мин

Таблица 7 – Матрица времен перемещения (минуты)

Из / В	v_0	v_1	v_2
v_0	—	40	15
v_1	40	—	20
v_2	15	20	—

Трассировка алгоритма при начале работы в 08:00 (для наглядности время приведено в минутах от полуночи; в реализации используются секунды Unix):

Выбор стартового узла. Узел v_0 имеет наиболее раннее окно (08:00), поэтому именно он выбирается стартовым: $\text{cur} = 0$, $\text{currentTime} = 480 + 30 = 510$ (08:30 плюс 30 мин выполнения, что соответствует 09:30 или 510 мин от полуночи).

Шаг 1. Проход 1. Для кандидата v_1 прибытие составляет $510 + 40 = 550$ (09:10). Окно v_1 задано как 600–720 (10:00–12:00), условие $550 \leq 720 - 20 = 700$ выполнено, время в пути $t = 40$ мин. Для кандидата v_2 прибытие составляет $510 + 15 = 525$ (08:45), окно 540–600 (09:00–10:00), условие $525 \leq 600 - 15 = 585$ выполнено, $t = 15$ мин. Поскольку v_2 ближе ($15 < 40$), выбирается $\text{next} = 2$. Ожидание открытия окна составляет $\max(0, 540 - 525) = 15$ мин, что даёт $\text{currentTime} = 525 + 15 + 15 = 555$ (09:15).

Шаг 2. Проход 1. Для оставшегося кандидата v_1 прибытие составляет $555 + 20 = 575$ (09:35), условие $575 \leq 700$ выполнено, $\text{next} = 1$. Ожидание равно $\max(0, 600 - 575) = 25$ мин, после чего $\text{currentTime} = 575 + 25 + 20 = 620$ (10:20).

Итоговый порядок: $v_0 \rightarrow v_2 \rightarrow v_1$. Суммарное ожидание: $15 + 25 = 40$ мин.

2.3.5 Оценка вычислительной сложности

Алгоритм выполняет n итераций основного цикла; на каждой итерации для каждого из n кандидатов выполняется одношаговый просмотр вперед (`causesWindowMiss`) по n оставшимся узлам, что дает сложность $O(n^3)$.

При $n = 30$ задачах алгоритм выполняет порядка $30^3 = 27\,000$ элементарных операций. Результаты бенчмаркинга, приведенные в таблице 11, показывают, что при $n \leq 30$ время работы алгоритма составляет единицы микросекунд, что на несколько порядков меньше задержки, вносимой получением матрицы расстояний от внешнего API.

2.3.6 Дополнительные ограничения маршрута

Помимо временных окон, реализована поддержка трёх типов дополнительных ограничений, задаваемых пользователем при запросе оптимизации.

Фиксированная начальная точка. Если пользователь указывает идентификатор начальной задачи, алгоритм начинает обход с соответствующего узла вместо автоматически выбранного по признаку наиболее раннего временного окна.

Фиксированная конечная точка. Если задана конечная задача, соответствующий узел исключается из стандартного процесса выбора следующей точки. После того как все остальные узлы посещены, конечный узел добавляется в маршрут последним.

Ограничения порядка. Пользователь вправе указать пары задач вида «А до В»: задача А должна быть выполнена строго раньше задачи В. При выборе следующего узла на каждом шаге алгоритм пропускает кандидатов, для которых не все предшественники уже посещены. Условие проверяется в

обоих проходах (Pass 1 и Pass 2), что гарантирует соблюдение ограничения независимо от наличия допустимых временных окон.

Ограничения передаются в алгоритм через структуру `Constraints` пакета `optimizer`. Преобразование идентификаторов задач в индексы графа выполняется на уровне бизнес-логики (`story.Optimize`) и не затрагивает реализацию алгоритма, что соответствует принципу единственной ответственности.

2.4 Интерфейс

На рисунках 4, 5 и 6 приведены экранные формы реализованного приложения. Рисунок 4 соответствует странице авторизации, рисунок 5 — главной странице до запуска оптимизации, рисунок 6 — ей же после построения маршрута.

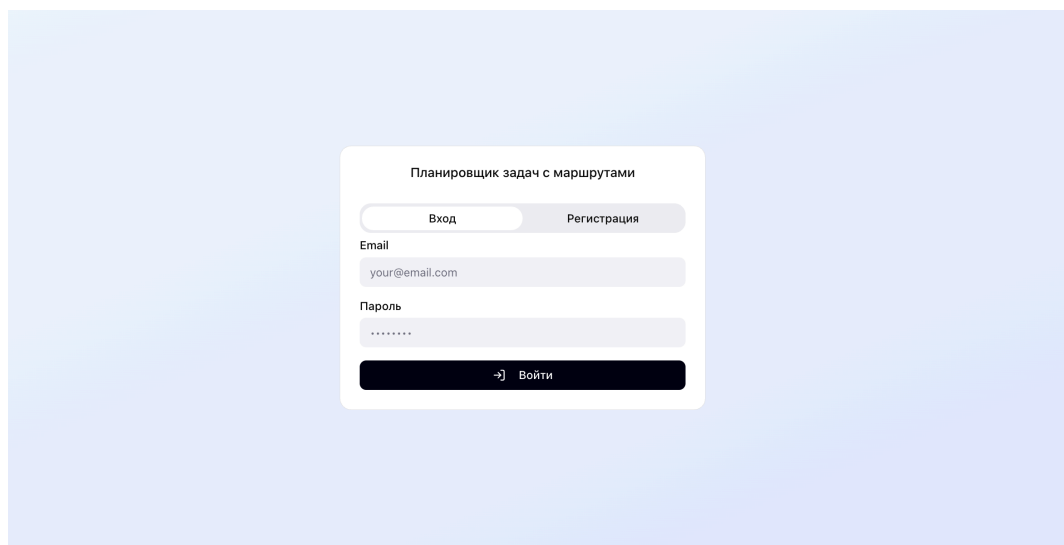


Рисунок 4 – Страница авторизации

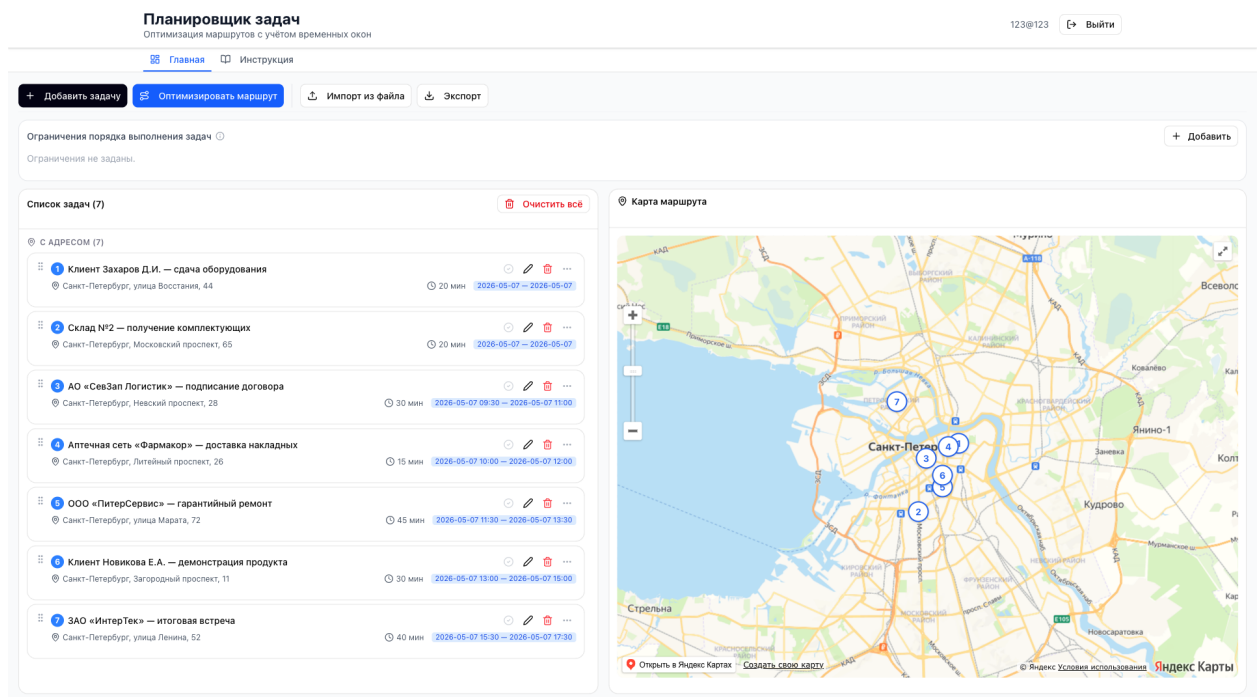


Рисунок 5 – Главная страница приложения до запуска оптимизации

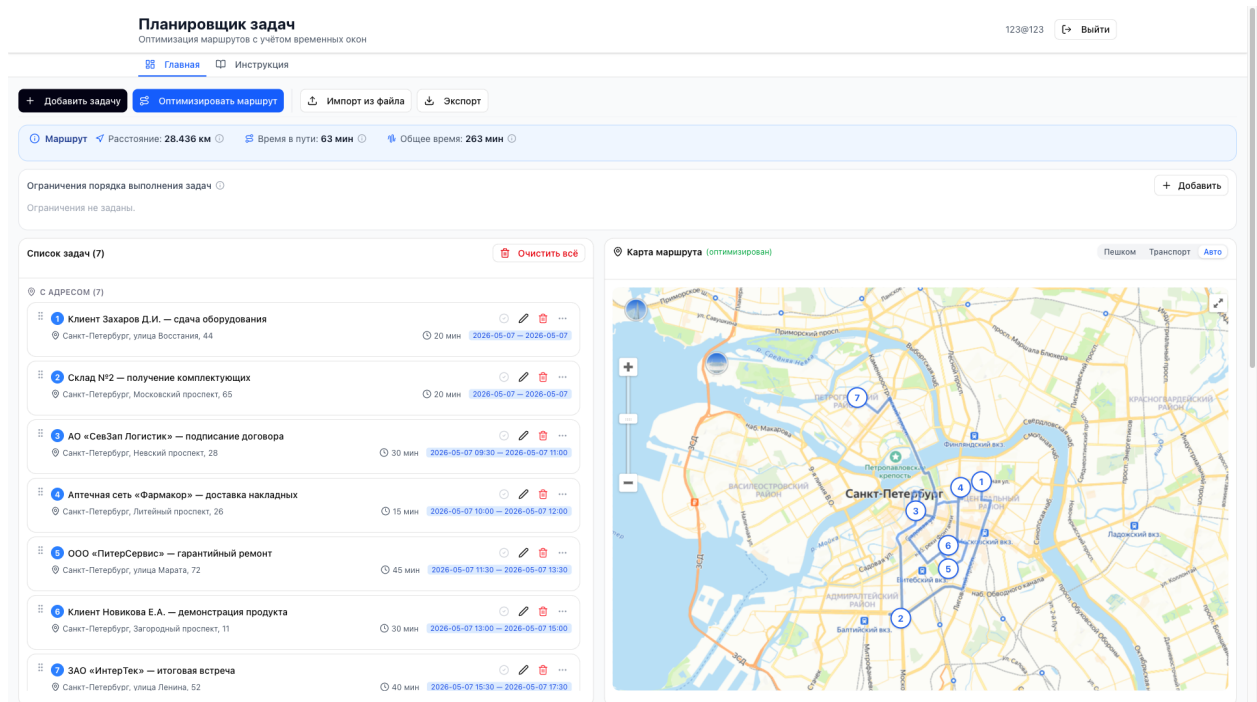


Рисунок 6 – Главная страница после построения маршрута

2.4.1 Структура и организация интерфейса

Главная страница разделена на две функциональные зоны (рисунок 5): слева — список задач, справа — карта с визуализацией маршрута. Размещение в пределах одного экрана позволяет немедленно оценивать влияние изменений в списке задач на итоговый маршрут без переключения между

представлениями. Список задач отображается в виде отдельных карточек с фиксированным набором полей (номер, название, адрес, длительность, временное окно). Карта реализована на платформе Яндекс Карты, что обеспечивает связь между порядком задач и их пространственным расположением.

После запуска оптимизации (рисунок 6) над списком появляется панель сводки маршрута с суммарным расстоянием, временем в пути и общим временем выполнения; на карте — линия маршрута с пронумерованными метками и переключатель способа передвижения; карточки перестраиваются в порядке посещения, рассчитанном алгоритмом.

Добавление задачи выполняется через модальное окно с логически сгруппированными полями: название, адрес, длительность, временное окно (дата и время начала, дата и время окончания). Поле адреса поддерживает автодополнение через сервис геокодирования Яндекса: выбор подсказки автоматически заполняет географические координаты. Навигация организована через верхнюю панель и включает две вкладки — «Главная» и «Инструкция»; в правой части шапки размещены идентификатор активной учетной записи и кнопка выхода из сессии.

2.4.2 Импорт и экспорт задач

Для ускорения ввода большого количества задач реализован импорт из файлов CSV и Excel (XLSX). Диалог импорта отображает распознанные строки, выделяет ошибки валидации (отсутствие названия, некорректный формат времени) и позволяет пользователю подтвердить или отклонить каждую строку перед сохранением. Импортированные задачи создаются пакетно через обработчик HTTP-запросов `POST /api/tasks/batch`.

Экспорт задач доступен в двух форматах: CSV (библиотека `raparparse` [50]) и Excel (библиотека `xlsx` [51]). Экспортируются

все текущие задачи пользователя с полями: название, адрес, длительность, дата и время временного окна.

2.4.3 Дополнительные элементы управления

В интерфейсе реализована возможность изменения порядка задач методом перетаскивания: даже после автоматической оптимизации пользователь может скорректировать последовательность вручную без повторного запуска алгоритма. Помимо этого, реализованы следующие элементы управления:

- **Выбор режима транспорта:** пользователь выбирает способ перемещения (автомобиль, пешком, общественный транспорт), что влияет на матрицу расстояний и визуализацию маршрута на карте. Для режима «общественный транспорт» расчёт времени приближается авто-режимом из-за ограничений API мультимаршрутизации, тогда как построение и отображение маршрута производится корректно для всех режимов;

- **Фиксация начальной и конечной точки:** через контекстное меню карточки задачи пользователь назначает точку старта или финиша маршрута;

- **Ограничения порядка:** пользователь задает пары «задача А до задачи В», которые алгоритм обязан соблюдать;

- **Обнаружение конфликтов:** интерфейс автоматически выявляет и подсвечивает задачи с некорректными или конфликтующими временными окнами как до оптимизации (взаимное перекрытие), так и после (фактическое опоздание);

- **Отметка выполнения:** задача может быть помечена как выполненная;

- **Клонирование задач:** дублирование существующей задачи для быстрого создания аналогичной записи.

2.4.4 Адаптивная вёрстка

Интерфейс реализован с поддержкой адаптивной вёрстки и сохраняет работоспособность на устройствах различных классов: настольные мониторы, планшеты и смартфоны. Адаптация выполнена средствами фреймворка Tailwind CSS [49] с применением контрольных точек ширины.

На широких экранах (рисунок 5) список задач и карта размещаются рядом в две колонки, что обеспечивает их одновременный обзор. При сужении области просмотра до планшета (рисунок 7) двухколоночная компоновка перестраивается в одноколоночную: карта сдвигается под список задач, а кнопки верхней панели инструментов располагаются в две строки. На экранах смартфонов (рисунок 8) текстовые подписи кнопок сворачиваются до иконок, а длинные названия задач усекаются с многоточием для сохранения целостности карточек.

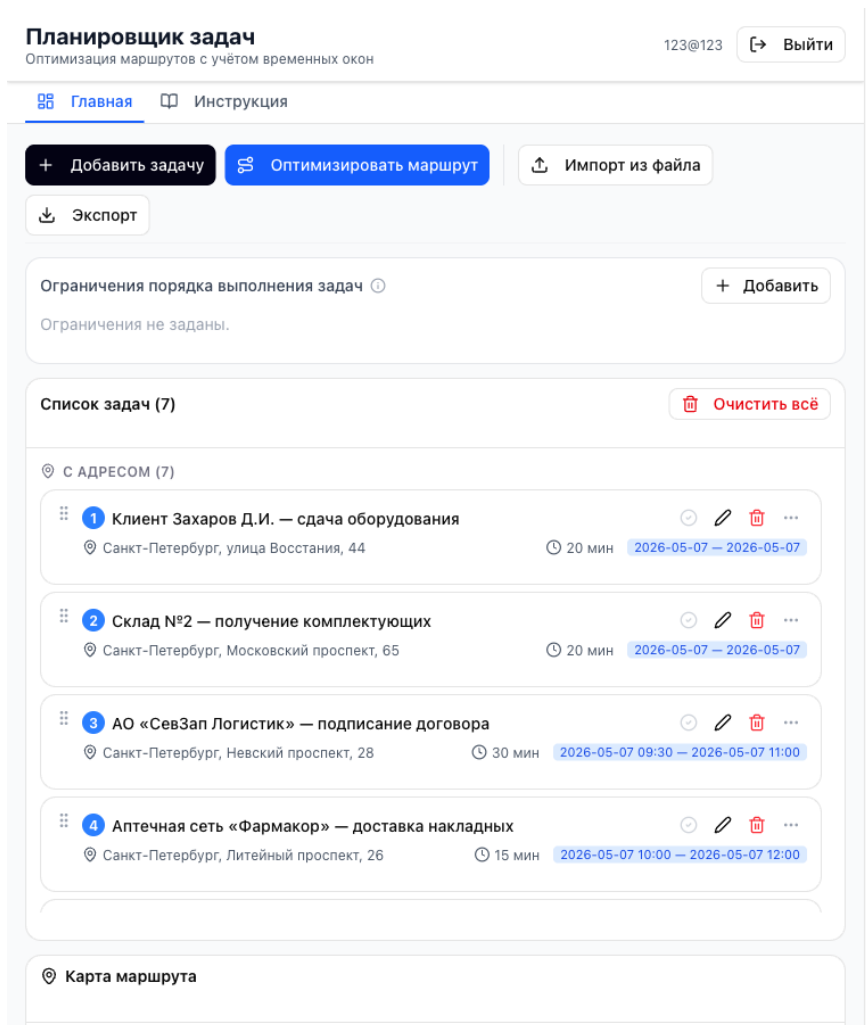


Рисунок 7 – Главная страница приложения на планшете

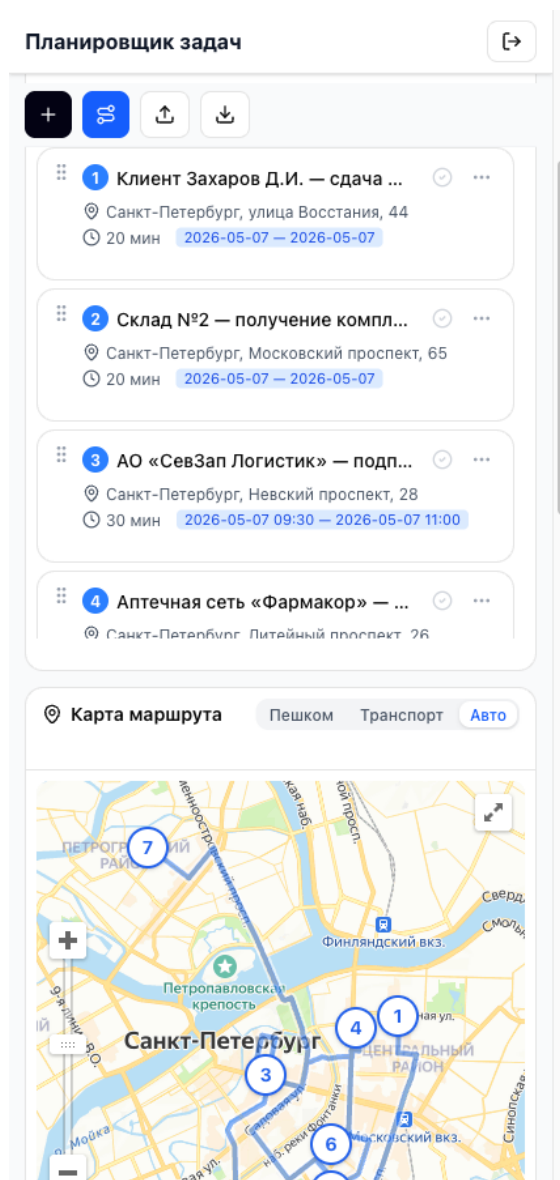


Рисунок 8 – Главная страница приложения на смартфоне после построения маршрута

2.5 Техническая реализация

2.5.1 Состав и объем программного кода

Приложение включает два независимых программных компонента: серверную часть на Go и клиентскую часть на TypeScript с библиотекой React. Серверная часть содержит около 54 файлов Go, организованных по пакетам в соответствии с принципами чистой архитектуры (доменные сущности и варианты использования, HTTP-обработчики, репозитории

PostgreSQL, провайдер матрицы расстояний) и пять SQL-миграций. Клиентская часть включает около 100 файлов TypeScript/TSX, из которых около 40 реализуют прикладную логику и компоненты страниц, а остальные — примитивы библиотеки пользовательского интерфейса на основе shadcn/ui [52] и Radix UI [53].

Наиболее значимые фрагменты кода приведены в приложениях: реализация алгоритма оптимизации маршрута — в приложении Б.1, HTTP-обработчики маршрутов — в приложении Б.2, клиент API с логикой обновления токена — в приложении Б.3, утилита построения матрицы расстояний — в приложении Б.4.

2.5.2 Требования к техническим средствам

Для развертывания и эксплуатации приложения предъявляются следующие требования к техническим средствам.

Для серверной части:

- процессор: одноядерный с тактовой частотой не ниже 1 ГГц (рекомендуется двухъядерный и более);
- оперативная память: не менее 512 МБ (рекомендуется 1 ГБ и более);
- дисковое пространство: не менее 1 ГБ для хранения образов Docker и данных PostgreSQL;
- сетевой интерфейс: соединение с интернетом для обращения к внешним сервисам (Яндекс Карты, OSRM).

Для клиентской части (рабочее место пользователя):

- устройство с современным веб-браузером и поддержкой JavaScript;
- соединение с интернетом для загрузки Яндекс Карт и обращения к API.

2.5.3 Требования к программному обеспечению

Для развертывания серверной части необходимы:

- операционная система Linux [46], macOS [47] или Windows [48] с поддержкой контейнеризации;
- Docker Engine версии 24.0 и выше;
- Docker Compose версии 2.0 и выше.

Для сборки и запуска клиентской части в режиме разработки необходимы Node.js [54] версии 18 и выше и менеджер пакетов `npm` [55] версии 8 и выше (либо `pnpm` версии 9 и выше). На стороне пользователя поддерживаются браузеры Google Chrome версии 90 и выше, Mozilla Firefox версии 88 и выше, Apple Safari версии 14 и выше, Microsoft Edge версии 90 и выше.

2.6 Руководство пользователя и администратора

2.6.1 Установка и настройка серверной части

Приложение состоит из двух независимых компонентов: серверной части (Go API + PostgreSQL) и клиентской части (React SPA). Серверная часть распространяется в виде Docker-образа и разворачивается через Docker Compose, что автоматически поднимает базу данных PostgreSQL, службу применения миграций и API-сервер. Клиентская часть собирается в набор статических файлов и размещается на веб-сервере (Nginx, Apache, Caddy и т. п.); пользователю достаточно современного веб-браузера.

Последовательность установки серверной части:

- а) Убедиться в наличии Docker (версии 24.0 и выше) и Docker Compose (версии 2.0 и выше).
- б) Перейти в директорию `backend/`.
- в) Создать файл `.env`, скопировав шаблон `.env.example`, и задать значения `POSTGRES_PASSWORD` и `APP_JWT_SECRET`; остальные параметры имеют значения по умолчанию.
- г) Выполнить команду:

```
docker compose up --build
```

После запуска Docker Compose последовательно поднимает контейнер PostgreSQL, дожидается готовности СУБД (`pg_isready`), применяет SQL-миграции через утилиту `migrate` и запускает API-сервер на порту 8080.

Конфигурация серверной части задается через переменные окружения; их перечень приведен в таблице 8.

Таблица 8 – Переменные окружения серверной части

Переменная	Значение по умолчанию	Описание
POSTGRES_DB	planner	Имя базы данных PostgreSQL
POSTGRES_USER	planner	Имя пользователя PostgreSQL
POSTGRES_PASSWORD	—	Пароль пользователя PostgreSQL
APP_HTTP_ADDR	:8080	Адрес и порт HTTP-сервера
APP_DB_DSN	—	Строка подключения к PostgreSQL
APP_JWT_SECRET	—	Секретный ключ для подписи JWT
APP_ACCESS_TTL_MIN	15	Время жизни access-токена (минуты)
APP_REFRESH_TTL_DAYS	30	Время жизни refresh-токена (дни)
APP_CORS_ORIGINS	http://localhost:5173	Допустимые источники CORS
APP_OSRM_BASE_URL	http://router.project-osrm.org	Базовый URL OSRM-сервиса

Обязательные переменные — `POSTGRES_PASSWORD` и `APP_JWT_SECRET`; значение `APP_DB_DSN` формируется автоматически из `POSTGRES_USER`, `POSTGRES_PASSWORD` и `POSTGRES_DB`. Для продуктивной эксплуатации `APP_JWT_SECRET` заменяется на криптографически стойкую случайную строку длиной не менее 32 символов, а `POSTGRES_PASSWORD` — на уникальный пароль доступа к базе данных.

2.6.2 Установка и настройка клиентской части

2.6.2.1 Режим разработки

Для запуска клиентского приложения в режиме разработки необходимо:

а) Убедиться в наличии Node.js версии 18 и выше, менеджера пакетов `pnpm` версии 8 и выше (либо `npm` версии 9 и выше).

б) Перейти в директорию `frontend/Task Planning App/`.

в) Создать файл `.env.local` и указать ключ Яндекс JS API:

```
VITE_YANDEX_GEOCODER_KEY=<ключ>
```

Ключ необходимо получить на портале разработчиков Яндекса, подключив сервис «JavaScript API и HTTP Геокодер». Активация ключа занимает до 15 минут после регистрации.

г) Установить зависимости и запустить сервер разработки:

```
pnpm install
```

```
pnpm dev
```

Приложение будет доступно по адресу `http://localhost:5173`. Все запросы к адресам вида `/api/*` автоматически проксируются на серверную часть по адресу `http://localhost:8080` через встроенный механизм Vite.

2.6.2.2 Сборка для продуктивной эксплуатации

Для получения оптимизированной статической сборки выполняется команда:

```
pnpm build
```

Собранные файлы размещаются в директории `dist/` и могут быть опубликованы на любом веб-сервере (Nginx, Apache, Caddy и других). При этом необходимо настроить перенаправление всех запросов на файл `index.html` для корректной работы клиентской маршрутизации, а также проксирование запросов к API на серверную часть.

2.6.3 Перечень возможных затруднений и способы их устранения

В таблице 9 приведены типичные проблемы, возникающие при установке и эксплуатации приложения, и способы их устранения.

Таблица 9 – Типичные проблемы и способы их устранения

Проблема	Способ устранения
API-сервер не запускается: ошибка подключения к базе данных	Убедиться, что контейнер <code>db</code> запущен и прошел проверку <code>healthcheck</code> ; проверить корректность строки <code>APP_DB_DSN</code>
Карта не отображается в браузере	Проверить наличие и корректность ключа <code>VITE_YANDEX_GEOCODER_KEY</code> в файле <code>.env.local</code> ; убедиться в доступности серверов Яндекс JS API
Геокодирование не возвращает результатов	Проверить лимиты использования API (до 1 000 запросов в сутки на бесплатном тарифе); при превышении — подождать до следующего дня или перейти на платный тариф
Порт 8080 или 5432 уже занят	Изменить проброс портов в <code>docker-compose.yml</code> (например, <code>"18080:8080"</code>) и соответственно обновить значение <code>APP_CORS_ORIGINS</code>
Изменения в <code>.env.local</code> не применяются	Перезапустить сервер разработки Vite: он не выполняет горячую перезагрузку переменных окружения
Ошибка при применении миграций	Проверить, что база данных создана и пользователь обладает правами на создание таблиц; при частичном применении выполнить откат командой <code>migrate . . . down</code> и повторно <code>up</code>

2.6.4 Краткое руководство пользователя

Для начала работы с приложением пользователю необходимо:

- а) Открыть приложение в браузере и зарегистрироваться, указав адрес электронной почты и пароль, либо войти в существующую учетную запись.
- б) На главной странице нажать кнопку добавления задачи. В открывшемся модальном окне ввести название задачи, адрес выполнения, продолжительность и временное окно (при необходимости).
- в) После добавления нескольких задач нажать кнопку «Оптимизировать маршрут». Приложение выполнит геокодирование адресов, получит матрицу расстояний и вычислит оптимальный порядок задач.
- г) Результат отображается одновременно в списке задач (в оптимальном порядке) и на карте (в виде маршрутной линии с пронумерованными метками).
- д) При необходимости пользователь может вручную изменить порядок задач методом перетаскивания карточек.
- е) Для удаления или редактирования задачи следует воспользоваться соответствующими элементами управления на карточке задачи.

2.6.5 Руководство администратора

Администратор системы несет ответственность за развертывание, поддержание работоспособности и обновление серверной части приложения.

Резервное копирование. Данные PostgreSQL хранятся в именованном томе Docker db_data. Для создания резервной копии следует использовать утилиту pg_dump:

```
docker exec <container_db> pg_dump -U planner planner > bac
```

Восстановление из резервной копии:

```
docker exec -i <container_db> psql -U planner planner < bac
```

Обновление приложения. При выходе новой версии необходимо пересобрать образы командой `docker compose up --build`, которая автоматически применит новые миграции базы данных перед запуском сервера.

Мониторинг. Состояние контейнеров отслеживается командой `docker compose ps`, журналы приложения — командой `docker compose logs api`.

Обеспечение безопасности. В продуктивной среде необходимо: заменить все пароли по умолчанию в `docker-compose.yml`, установить значение `APP_JWT_SECRET` равным криптографически стойкой случайной строке, ограничить доступ к портам 5432 и 8080 на уровне межсетевого экрана, предоставив внешний доступ только к порту 8080 (или разместив приложение за обратным прокси-сервером с поддержкой TLS).

2.7 Выводы

Спроектирована модель данных, поддерживающая хранение задач, маршрутов, временных окон и пользовательских ограничений.

Серверная часть построена по принципам чистой архитектуры с разделением слоев домена, вариантов использования и инфраструктуры; реализован алгоритм NNH-TW с одношаговым просмотром вперед и поддержкой фиксированных начальной и конечной точек маршрута, а также попарных ограничений порядка посещения.

Разработан клиентский интерфейс с интерактивной картой и средствами ручного управления маршрутом; подготовлена процедура развертывания приложения через Docker Compose, обеспечивающая воспроизводимость опытной эксплуатации.

3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ

3.1 Анализ внедрения продукта

В первой главе сформулированы цель разработки и перечень функциональных и нефункциональных требований к системе. Сопоставление этих требований с результатами реализации приведено в таблице 10.

Таблица 10 – Соответствие реализованной системы поставленным требованиям

Требование	Результат
Создание, редактирование и удаление задач	Реализовано
Хранение географических координат	Реализовано
Получение матрицы расстояний через внешние API	Реализовано (Яндекс мультимаршрутизатор на клиенте, OSRM как резервный, кэш в памяти приложения)
Вычисление оптимальной последовательности с учетом временных окон	Реализовано
Расчет времени прибытия, ожидания и завершения	Реализовано
Визуализация маршрута на карте	Реализовано (Яндекс Карты)
Экспорт задач в форматах CSV и XLSX	Реализовано
Импорт задач из CSV и XLSX	Реализовано
Дополнительные ограничения маршрута (начальная/конечная точка, порядок)	Реализовано
Выбор режима транспорта	Реализовано (авто, пешком, общественный транспорт; в последнем случае расчёт времени приближается авто-режимом, визуализация выполняется корректно)
Обнаружение конфликтов временных окон	Реализовано
Время ответа сервера не более 2 секунд	Выполнено
Поддержка современных браузеров	Выполнено
Устойчивость к повторным запросам API за счет кэширования	Выполнено
Масштабируемость до 100 одновременных пользователей	Подтверждено нагрузочным тестированием: при 100 VU p95 = 1370 мс < 2000 мс, доля ошибок — 0 %

Все функциональные и нефункциональные требования, поставленные в первой главе, выполнены.

3.2 Аспекты отказоустойчивости, надежности и производительности

3.2.1 Устойчивость к отказам внешних зависимостей

Ключевым риском при интеграции с внешним картографическим сервисом является его деградация или временная недоступность. На стороне сервера этот риск снижает кэш матрицы расстояний (см. раздел «Кэширование внешних запросов» второй главы): при наличии записи в кэше повторное обращение к OSRM не производится, что уменьшает суммарную задержку оптимизации и исключает зависимость от мгновенной доступности маршрутизатора.

На стороне клиента загрузка Яндекс JS API выполняется через утилиту `loadYandexMaps()`, которая инжектирует `<script>`-тег динамически при первом обращении и возвращает один и тот же Promise при последующих вызовах. Такой подход предотвращает дублирующую загрузку скрипта при одновременном обращении нескольких компонентов и исключает состояние гонки при инициализации API.

3.2.2 Надежность аутентификации

Аутентификация реализована по схеме JWT [12] с двумя токенами: краткосрочным `access`-токеном (время жизни — 15 минут) и долгосрочным `refresh`-токеном (время жизни — 30 дней). Применение пары `access/refresh`-токенов соответствует общему подходу к делегированной авторизации, описанному в [28]. Токены обновления хранятся в базе данных в виде хэша SHA-256 [61], что исключает их компрометацию при несанкционированном доступе к СУБД; данная мера согласуется с практическими рекомендациями по защите сервисов, использующих JWT [29]. Пароли поль-

зователей хэшируются алгоритмом `bcrypt` [13] с адаптивным количеством раундов.

На клиентской стороне реализован механизм автоматического обновления токена при получении ответа с кодом 401. Корректность этого механизма проверяется модульными тестами API-клиента с перехватом HTTP-запросов через `Mock Service Worker (MSW)` [56]. Для исключения множественных одновременных запросов на обновление применяется дедупликация: при первом истечении токена запускается единственный запрос обновления, остальные запросы ожидают его завершения через общий `Promise`. После успешного обновления исходный запрос повторяется автоматически — пользователь не замечает прерывания сессии.

Сессия пользователя сохраняется в `localStorage` под ключом `planner.auth.session`. При закрытии и повторном открытии браузера приложение автоматически восстанавливает состояние аутентификации без повторного ввода учетных данных.

Диаграммы последовательности, отражающие три сценария взаимодействия компонентов системы аутентификации, представлены на рисунках 9, 10 и 11.

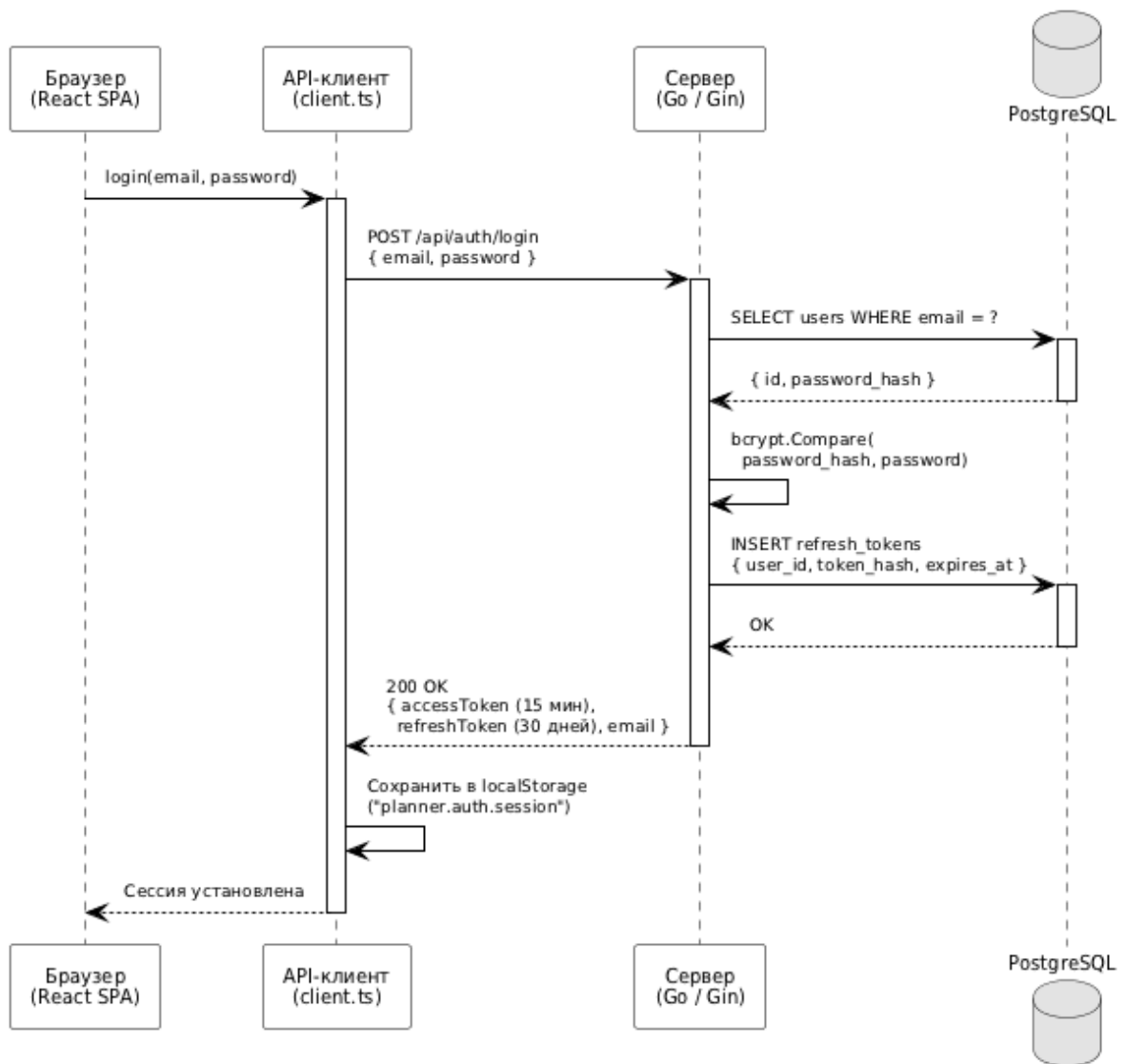


Рисунок 9 – Диаграмма последовательности: сценарий входа в систему

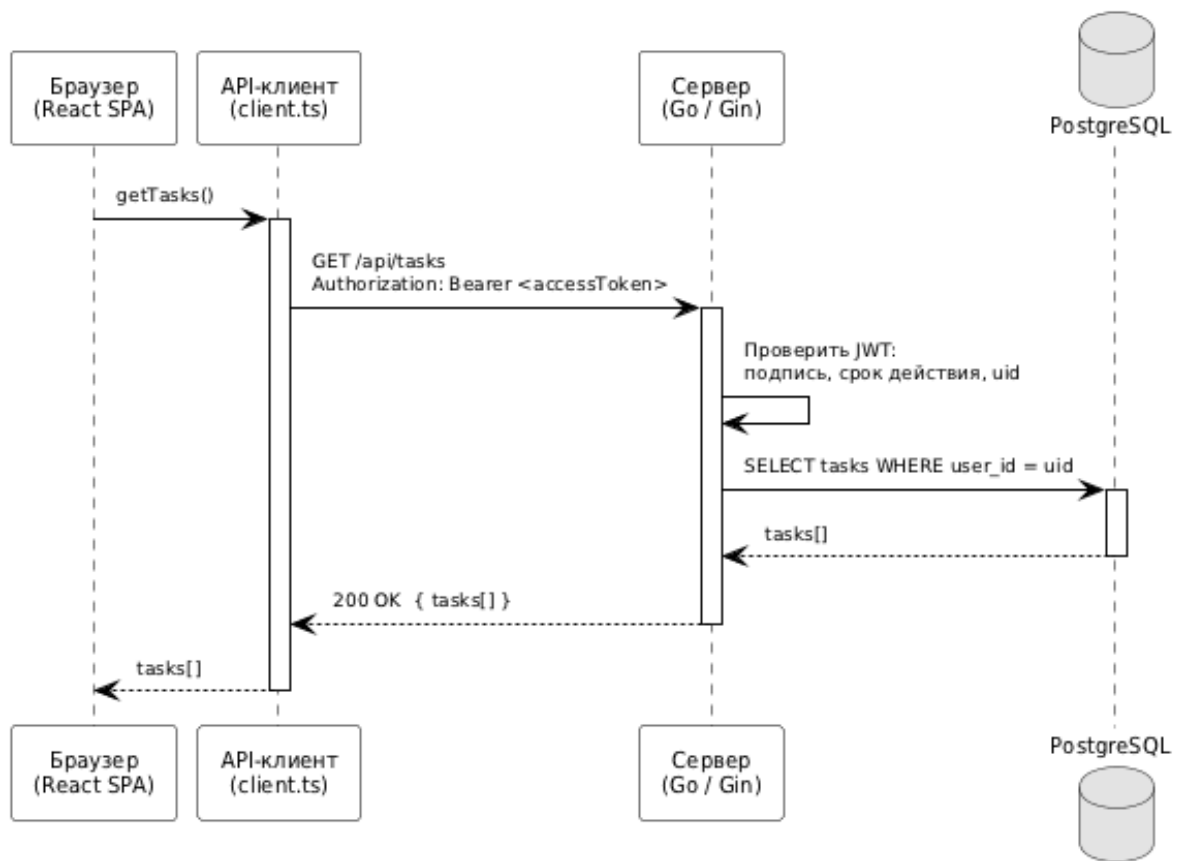


Рисунок 10 – Диаграмма последовательности: запрос с валидным токеном

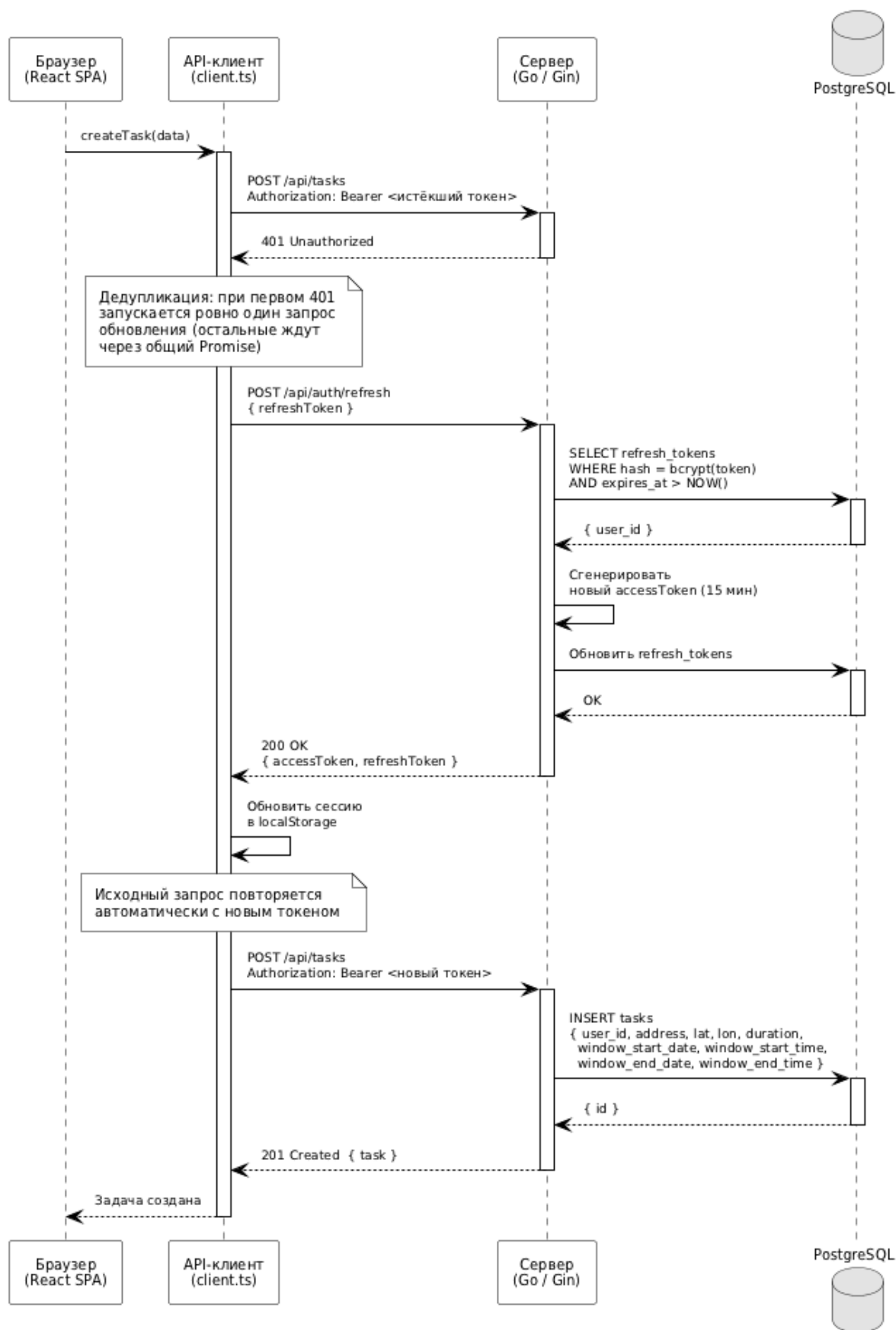


Рисунок 11 – Диаграмма последовательности:
автоматическое обновление токена при ответе 401

3.2.3 Надежность инфраструктуры

При развертывании с использованием Docker Compose [5] зависимость между контейнерами управляется через механизм `healthcheck`: сервис применения миграций и API-сервер запускаются только после того, как контейнер базы данных PostgreSQL подтвердит готовность к приему соединений командой `pg_isready`. Это исключает ситуацию, при которой API-сервер стартует до завершения инициализации СУБД.

Схема базы данных управляется инструментом `migrate/migrate` [57], запускаемым автоматически при каждом старте стека. Применение миграций является идемпотентным: повторный запуск `migrate up` при уже актуальной схеме не вносит изменений. Это позволяет безопасно перезапускать стек без ручного контроля состояния миграций.

3.2.4 Производительность алгоритма оптимизации

Для оценки производительности алгоритма NNH-TW использован встроенный инструмент бенчмаркинга языка Go (`testing.Benchmark`). Измерения проводились на рабочей станции с процессором Apple M1 (8 ядер) и 16 ГБ оперативной памяти под управлением операционной системы macOS 15. Для каждого значения n выполнялось не менее 50 000 итераций; в таблице приведено среднее время одного вызова. Результаты представлены в таблице 11.

Таблица 11 – Производительность алгоритма NNH-TW в зависимости от числа задач

Число задач (n)	Запросов к API матрицы ($n(n - 1)$)	Время NNH-TW (Go)	Узкое место
5	20	0,36 мкс	HTTP-запросы к мультимаршрутизатору Яндекс JS API
10	90	1,12 мкс	HTTP-запросы к мультимаршрутизатору Яндекс JS API
20	380	3,66 мкс	HTTP-запросы к мультимаршрутизатору Яндекс JS API
30	870	6,96 мкс	HTTP-запросы к мультимаршрутизатору Яндекс JS API
50	2450	24,54 мкс	HTTP-запросы к мультимаршрутизатору Яндекс JS API

Из таблицы 11 следует, что узким местом системы является не сам алгоритм оптимизации (сложность $O(n^3)$, выполнение при $n \leq 30$ за единицы-десятки микросекунд), а получение матрицы расстояний от внешнего API. При $n = 20$ задачах число параллельных запросов к Яндекс JS API составляет $20 \times 19 = 380$, однако эти запросы выполняются параллельно на стороне клиента через `Promise.all`, что сводит суммарную задержку к времени наиболее медленного из них; доминирующая задержка определяется сетевыми условиями и временем ответа внешнего API.

Кэш матрицы расстояний в памяти приложения снижает нагрузку на внешний API при повторных оптимизациях: если пара точек уже была обработана ранее, расстояние возвращается из кэша без обращения к OSRM.

3.2.5 Нагрузочное тестирование

Для проверки нефункционального требования о масштабируемости до 100 одновременных пользователей проведено нагрузочное тестирование REST API с использованием инструмента k6 [34]; Тест воспроизводил типовой сценарий работы пользователя: аутентификация, получение списка задач (GET /api/tasks) и создание новой задачи (POST /api/tasks). Каждый сценарий выполнялся в течение 60 секунд при фиксированном числе виртуальных пользователей (VU). Тестовый стенд: ноутбук с процессором Apple M2 (8 ядер), 16 ГБ оперативной памяти, серверный стек запущен локально через Docker Compose. Критерии успешности: 95-й перцентиль времени ответа менее 2000 мс, доля ошибочных HTTP-ответов менее 1 %.

Результаты нагрузочного тестирования приведены в таблице 12.

Таблица 12 – Результаты нагрузочного тестирования REST API

VU	Запросов	avg, мс	p90, мс	p95, мс	Ошибки
10	1500	69	133	159	0 %
50	5484	219	475	577	0 %
100	5799	720	1250	1370	0 %

Все три сценария выдержали установленные критерии: ни один запрос не завершился ошибкой, а 95-й перцентиль времени ответа при 100 одновременных пользователях составил 1370 мс при допустимом пороге в 2000 мс. Нефункциональное требование масштабируемости подтверждено экспериментально.

3.3 Тестирование

3.3.1 Тестирование алгоритма оптимизации

Стратегия тестирования следует общим принципам отбора тест-кейсов на основе классов эквивалентности, граничных значений и ветвлений алгоритма [27, 43]. Наиболее критичным компонентом с точки зрения коррект-

ности является алгоритм NNH-TW. Для его проверки разработано 35 тест-кейсов, охватывающих все ветви алгоритма. Основные тест-кейсы приведены в таблице 13.

Таблица 13 – Тест-кейсы алгоритма NNH-TW

Название теста	Проверяемое условие
TestOptimize_TwoNodesNoWindows	Граф из 2 узлов без временных ограничений; оба посещены ровно один раз
TestOptimize_ThreeNodesNearestNeighbor	3 узла; ближайший сосед выбирается корректно (порядок 0→1→2)
TestOptimize_TimeWindowEarliestFirst	Стартовый узел выбирается по наиболее раннему временному окну
TestOptimize_WaitBeforeWindow	Ожидание перед открытием окна (TotalWaitSec > 0)
TestOptimize_FallbackPass	Проход 2: нода выбирается без учета окна при отсутствии допустимых узлов
TestOptimize_AggregateStats	Корректность агрегатных показателей (расстояние, время, сервис)
TestOptimize_OneNode	Граф из одного узла; order = [0]
TestOptimize_StartTime0800	Начало работы в 08:00; оба узла посещены
TestOptimize_SymmetricMatrix	Симметричная матрица из 4 узлов; каждый посещен ровно один раз
TestOptimize_UnreachablePairs	«Недостижимые» пары (99 999 сек); все узлы посещены через Pass 2
TestOptimize_EmptyGraph	Пустой граф; order = [] без ошибок
TestFeasible_*(5 случаев)	Граничные значения функции feasible(): нет ограничений, в пределах окна, до открытия, после закрытия, на границе
TestOptimize_FixedStartNode	Фиксированная начальная точка: маршрут начинается с указанного узла
TestOptimize_FixedEndNode	Фиксированная конечная точка: маршрут завершается указанным узлом
TestOptimize_FixedStartAndEnd	Одновременное задание начальной и конечной точек
TestOptimize_PrecedenceConstraint	Ограничение предшествования: узел A посещается раньше узла B
TestOptimize_AllVisitedWithConstraints	Все узлы посещены при наличии комплексных ограничений

Продолжение таблицы 13

Название теста	Проверяемое условие
TestOptimize_LookAheadPreventsWindowMiss	Просмотр вперед предотвращает пропуск узла с жестким дедлайном
TestOptimize_TightDeadlineTiebreaker	Узел с более срочным дедлайном выбирается при равном времени завершения
TestOptimize_Timings* (4 случая)	Корректность таймингов остановок: длина, стартовый узел, ожидание, длительность обслуживания
TestStartNode_* (3 случая)	Выбор стартового узла: раннее окно, отсутствие окон, учет предшественников

3.3.2 Тестирование бизнес-логики серверной части

Бизнес-логика тестируется с использованием репозиториев-заглушек, генерируемых пакетом `testify/mock` [58], что позволяет проверять варианты использования в изоляции от базы данных. Покрываются следующие компоненты:

- `auth/story` — регистрация, вход, выход, обновление токена;
- `task/story` — создание, редактирование, удаление задачи, проверка принадлежности задачи пользователю;
- `route/story` — запуск оптимизации и сохранение единственного маршрута пользователя.

3.3.3 Тестирование клиентской части

Клиентская часть тестируется средствами `vitest` [59] — среды тестирования на базе `Vite` [22], библиотеки `@testing-library/react` [60] и `msw` (Mock Service Worker) [56] — для перехвата HTTP-запросов в тестах. Ключевые тест-кейсы:

— Логика обновления токена: при ответе 401 запускается ровно один запрос обновления (дедупликация через общий Promise); после успешного обновления исходный запрос повторяется;

— Утилита построения матрицы расстояний: формирование $n \times (n - 1)$ запросов к мультимаршрутизатору Яндекс JS API.

3.4 Внедрение и сопровождение программного продукта

Приложение является веб-приложением с клиент-серверной архитектурой; конечный пользователь взаимодействует с ним через стандартный веб-браузер без установки дополнительного программного обеспечения. Серверная часть распространяется в виде исходного кода и Dockerfile и разворачивается через Docker Compose; клиентская часть собирается в набор статических файлов командой `pnpm build` и публикуется на любом веб-сервере. Подробные процедуры установки, конфигурирования, обновления, резервного копирования и устранения типичных затруднений приведены в «Руководстве пользователя и администратора» второй главы. Перед переходом в продуктивную эксплуатацию обязательно меняются учетные данные СУБД, устанавливается криптографически стойкое значение `APP_JWT_SECRET` и задается корректный `APP_CORS_ORIGINS`.

3.4.1 Ограничения и направления развития

В текущей реализации хранение географических координат задач в базе данных осуществляется в соответствии с условиями бесплатного тарифа Яндекс JS API [19], которые допускают это для учебных и некоммерческих проектов; для коммерческого использования потребуется переход на платную лицензию. Суточный лимит бесплатного тарифа составляет 1 000 запросов к геокодеру и 25 000 запросов к JavaScript API; при превышении указан-

ных лимитов рекомендуется переход на коммерческий тариф либо интеграция с альтернативным геокодером.

Перспективные направления развития системы:

- реализация фоновых уведомлений о наступлении временного окна следующей задачи;
- поддержка командной работы с общим пулом задач;
- интеграция с внешними календарями (Google Calendar, Яндекс Календарь);
- расширение алгоритма оптимизации метаэвристиками (муравьиный алгоритм, генетический алгоритм) для повышения качества решений при большом числе задач;
- учет реального дорожного трафика при построении матрицы расстояний.

3.5 Выводы

Реализованная система соответствует функциональным и нефункциональным требованиям, сформулированным в первой главе.

Архитектурные решения — кэширование матрицы расстояний, хранение паролей и токенов обновления в виде хэшей, автоматическое восстановление сессии — обеспечивают отказоустойчивость и надежность работы. Стратегия тестирования охватывает алгоритм оптимизации (35 тест-кейсов с проверкой временных окон, просмотра вперед и попарных ограничений порядка), бизнес-логику серверной части (тесты через заглушки `testify/mock`) и клиентский API-клиент (тесты на стеке `vitest` с `MSW`).

По результатам нагрузочного тестирования REST API выдерживает 100 одновременных пользователей без ошибок при 95-м процентиле времени ответа 1370 мс, что подтверждает выполнение требования масштабируемости.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы разработано веб-приложение для автоматизированного планирования маршрутов с учетом временных окон, географического положения точек и длительности выполнения задач.

В результате работы решены следующие задачи:

1. Проведен анализ предметной области: выявлены ограничения существующих приложений для управления задачами и навигационных сервисов, обоснована актуальность разработки.

2. Исследованы алгоритмические подходы к решению задачи маршрутизации с временными окнами; выбрана и реализована эвристика ближайшего соседа с одношаговым просмотром вперед (NNH-TW) с временной сложностью $O(n^3)$.

3. Разработана архитектура программной системы на основе принципов чистой архитектуры с разделением на слои: доменная логика, варианты использования, HTTP-доставка и репозитории базы данных.

4. Реализована серверная часть на языке Go с использованием фреймворка Gin, предоставляющая REST API для управления задачами, маршрутами и аутентификацией пользователей. Алгоритм оптимизации поддерживает фиксацию начальной и конечной точки маршрута, а также попарные ограничения порядка посещения.

5. Разработан пользовательский интерфейс на React с отображением задач в виде карточек, интерактивной картой Яндекс JS API, выбором режима транспорта, ручным упорядочиванием методом перетаскивания, импортом задач из CSV и Excel, экспортом в CSV и XLSX и визуальным обнаружением конфликтов временных окон.

6. Реализована интеграция с сервисом «Яндекс JavaScript API и HTTP Геокодер» версии 2.1 для геокодирования адресов с автодополнением и ви-

зуализации маршрута, а также с OSRM в качестве резервного источника матрицы расстояний.

7. Проведено тестирование решения: 35 модульных тестов алгоритма оптимизации, тесты бизнес-логики и HTTP-обработчиков серверной части, тесты клиентского API-клиента.

8. Выполнена оценка производительности и масштабируемости: нагрузочное тестирование подтвердило, что при 100 одновременных пользователях 95-й перцентиль времени ответа составляет 1370 мс при допустимом пороге 2000 мс и нулевой доле ошибочных ответов.

Поставленная цель достигнута: разработанное веб-приложение объединяет управление задачами, автоматическую оптимизацию маршрута с учетом временных окон и визуализацию на карте — сочетание, которое отсутствует у рассмотренных аналогов и недоступно частному пользователю без коммерческой подписки. По сравнению с менеджерами задач (Todoist, Microsoft To Do) система дополнительно реализует геокодирование адресов и оптимизацию порядка посещения; по сравнению с навигационными сервисами (Google Maps, Яндекс Карты) — учет временных окон и длительности обслуживания; по сравнению с логистическими платформами (Routific, Яндекс Маршрутизация) — доступность для индивидуального пользователя без подписки.

Практическая значимость работы заключается в предоставлении частным пользователям, специалистам на выезде, курьерам малого бизнеса и индивидуальным предпринимателям единого инструмента планирования маршрутов с временными окнами, не требующего коммерческой подписки. Веб-форма распространения и контейнеризация на основе Docker Compose обеспечивают воспроизводимое развертывание на типовом серверном оборудовании и не предъявляют требований к рабочему месту пользователя помимо современного веб-браузера.

Перспективные направления развития:

- поддержка командной работы с общим пулом задач и распределенной ответственностью;
- интеграция с внешними календарями (Google Calendar, Яндекс Календарь) для двунаправленной синхронизации задач;
- реализация фоновых уведомлений о наступлении временных окон ближайших задач;
- внедрение метаэвристик (муравьиный алгоритм, генетический алгоритм) для повышения качества маршрутов при большом числе задач — интерфейс `Optimizer` в коде серверной части рассчитан на такую замену;
- учет реального дорожного трафика при формировании матрицы расстояний.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бондаренко Т. В., Гарибов А. И., Федотов Е. А. Решение задачи планирования маршрутов с учетом временных окон [Электронный ресурс] // Инженерный вестник Дона. — 2020. — № 5 (65). — URL: <https://cyberleninka.ru/article/n/reshenie-zadachi-planirovaniya-marshrutov-s-uchyotom-vremennyh-okon> (дата обращения: 27.02.2026).
2. The Go Programming Language [Электронный ресурс]. — Режим доступа: <https://go.dev/>, свободный (дата обращения: 15.03.2026).
3. PostgreSQL 16 Documentation [Электронный ресурс]. — The PostgreSQL Global Development Group, 2026. — Режим доступа: <https://www.postgresql.org/docs/16/index.html>, свободный (дата обращения: 15.03.2026).
4. React — The library for web and native user interfaces [Электронный ресурс]. — Режим доступа: <https://react.dev/>, свободный (дата обращения: 15.03.2026).
5. Docker: Accelerated Container Application Development [Электронный ресурс]. — Режим доступа: <https://www.docker.com/>, свободный (дата обращения: 15.03.2026).
6. Relog — сервис оптимизации маршрутов [Электронный ресурс]. — Режим доступа: <https://relog.ru/>, свободный (дата обращения: 20.03.2026).
7. Яндекс Маршрутизация — сервис автоматического построения маршрутов [Электронный ресурс]. — Режим доступа: <https://routing.yandex.ru/>, свободный (дата обращения: 20.03.2026).
8. Spoke — Route Optimization Software [Электронный ресурс]. — Режим доступа: <https://www.spokeapp.io/>, свободный (дата обращения: 20.03.2026).
9. Гвоздев Л. Р., Медведева Т. А. Решение задачи маршрутизации транспортных средств с временными окнами с помощью алгоритма муравьиных колоний [Электронный ресурс] // Молодой исследователь Дона. —

2022. — № 3 (36). — URL: <https://cyberleninka.ru/article/n/reshenie-zadachi-marshrutizatsii-transportnyh-sredstv-s-vremennymi-oknami-s-pomoschyu-algoritma-muravinyh-koloniya> (дата обращения: 01.03.2026).

10. Alessandretti L., Szell M. Urban Mobility // *Compendium of Urban Complexity*. — Cham: Springer, 2025. — P. 57–76. — DOI: 10.1007/978-3-031-82666-5_4.

11. Saioc G.-V., Shirchenko D., Chabbi M. Unveiling and Vanquishing Goroutine Leaks in Enterprise Microservices: A Dynamic Analysis Approach // *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering: Companion (ICSE-Companion 2024)*. — New York: ACM, 2024. — P. 314–325. — DOI: 10.1145/3639478.3640033.

12. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT): RFC 7519 [Электронный ресурс]. — Internet Engineering Task Force, 2015. — Режим доступа: <https://www.rfc-editor.org/rfc/rfc7519>, свободный (дата обращения: 09.04.2026).

13. Provos N., Mazieres D. A Future-Adaptable Password Scheme // *Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference*. — Monterey: USENIX Association, 1999. — P. 81–91.

14. Мартин Р. К. Чистая архитектура: искусство разработки программного обеспечения. — Санкт-Петербург: Питер, 2018. — 352 с. — (Серия «Библиотека программиста»). — ISBN 978-5-4461-0772-8.

15. Held M., Karp R. M. A Dynamic Programming Approach to Sequencing Problems // *Journal of the Society for Industrial and Applied Mathematics*. — 1962. — Vol. 10, No. 1. — P. 196–210. — DOI: 10.1137/0110015.

16. Гладков Л. А., Курейчик В. В., Курейчик В. М. Генетические алгоритмы: учебник / под ред. В. М. Курейчика. — 2-е изд., испр. и доп. — Москва: ФИЗМАТЛИТ, 2010. — 368 с. — ISBN 978-5-9221-0510-1.

17. Project OSRM — Open Source Routing Machine [Электронный ресурс]. — Режим доступа: <https://project-osrm.org/>, свободный (дата обращения: 01.03.2026).

ния: 11.04.2026).

18. Luxen D., Vetter C. Real-time Routing with OpenStreetMap Data // Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (GIS '11). — New York: ACM, 2011. — P. 513–516. — DOI: 10.1145/2093973.2094062.

19. Яндекс JavaScript API и HTTP Геокодер [Электронный ресурс]. — Режим доступа: <https://yandex.ru/dev/jsapi-v2-1/doc/ru/>, свободный (дата обращения: 12.04.2026).

20. Gin Web Framework [Электронный ресурс]. — Режим доступа: <https://gin-gonic.com/>, свободный (дата обращения: 11.04.2026).

21. TypeScript: JavaScript With Syntax For Types [Электронный ресурс]. — Режим доступа: <https://www.typescriptlang.org/>, свободный (дата обращения: 11.04.2026).

22. Vite: Next Generation Frontend Tooling [Электронный ресурс]. — Режим доступа: <https://vite.dev/>, свободный (дата обращения: 11.04.2026).

23. Bogner J., Merkel M. To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub // Proceedings of the 19th International Conference on Mining Software Repositories (MSR 2022). — New York: ACM, 2022. — P. 658–669. — DOI: 10.1145/3524842.3528454.

24. Liu F., Lu C., Gui L., Zhang Q., Tong X., Yuan M. Heuristics for Vehicle Routing Problem: A Survey and Recent Advances [Электронный ресурс]. — 2023. — Режим доступа: <https://arxiv.org/abs/2303.04147>, свободный (дата обращения: 12.04.2026).

25. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — Москва: Вильямс, 2013. — 1328 с. — ISBN 978-5-8459-1794-2.

26. ГОСТ 19.701-90 (ИСО 5807-85). Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения

условные и правила выполнения. — Москва: Стандартинформ, 1992. — 26 с.

27. Куликов С. С. Тестирование программного обеспечения. Базовый курс. — 2-е изд. — Минск: Четыре четверти, 2017. — 312 с. — ISBN 978-985-7081-77-0.

28. Hardt D. The OAuth 2.0 Authorization Framework: RFC 6749 [Электронный ресурс]. — Internet Engineering Task Force, 2012. — Режим доступа: <https://www.rfc-editor.org/rfc/rfc6749>, свободный (дата обращения: 26.04.2026).

29. Девицына С. Н., Пилькевич П. В., Удод Е. В. Способы улучшения защищенности сервисов, использующих JWT-токены // Экономика. Информатика. — 2023. — Т. 50, № 1. — С. 144–151. — DOI: 10.52575/2687-0932-2023-50-1-144-151.

30. Кудухов А. Н. К вопросу о применении алгоритмов замещения LRU в подсистемах кэширования информационных систем промышленных предприятий [Электронный ресурс] // Перспективы развития информационных технологий. — 2014. — URL: <https://cyberleninka.ru/article/n/k-voprosu-o-primenenii-algoritmov-zamescheniya-lru-v-podsistemah-keshirovaniya-informatsionnyh-sistem-promyshlennyh-predpriyatiy> (дата обращения: 26.04.2026).

31. Бронникова Е. М. Использование информационных технологий в тайм-менеджменте [Электронный ресурс] // Научные труды Вольного экономического общества России. — 2010. — URL: <https://cyberleninka.ru/article/n/ispolzovanie-informatsionnyh-tehnologiy-v-taymmenedzhmente> (дата обращения: 26.04.2026).

32. Шкодинский С. В., Козлова А. Р. Ключевые аспекты логистики последней мили в онлайн-торговле // Прикладные экономические исследования. — 2025. — № 5. — С. 223–228. — DOI: 10.47576/2949-1908.2025.5.5.026.

33. Necula R., Breaban M., Raschip M. Tackling Dynamic Vehicle Routing Problem with Time Windows by means of Ant Colony System // Proceedings of the 2017 IEEE Congress on Evolutionary Computation (CEC 2017). — Donostia:

IEEE, 2017. — P. 2480–2487. — DOI: 10.1109/CEC.2017.7969606.

34. Grafana Labs. k6: Modern load testing for developers [Электронный ресурс]. — Режим доступа: <https://k6.io/>, свободный (дата обращения: 12.04.2026).

35. Todoist: Organize your work and life, finally [Электронный ресурс]. — Режим доступа: <https://todoist.com/pricing>, свободный (дата обращения: 12.04.2026).

36. Microsoft To Do: Tasks, To-Do lists & Reminders [Электронный ресурс]. — Режим доступа: <https://to-do.microsoft.com/>, свободный (дата обращения: 12.04.2026).

37. Google Maps [Электронный ресурс]. — Режим доступа: <https://maps.google.com/>, свободный (дата обращения: 12.04.2026).

38. Google Maps Help. Getting directions and showing routes [Электронный ресурс]. — Режим доступа: <https://support.google.com/maps/answer/144339>, свободный (дата обращения: 12.04.2026).

39. Routific: Intelligent Route Optimization Software [Электронный ресурс]. — Режим доступа: <https://routific.com/pricing>, свободный (дата обращения: 12.04.2026).

40. HashiCorp. golang-lru: Golang LRU cache [Электронный ресурс]. — Режим доступа: <https://github.com/hashicorp/golang-lru>, свободный (дата обращения: 12.04.2026).

41. Авдошин С. М., Береснева Е. Н. Эвристические методы конструирования маршрута для решения задачи маршрутизации с ограничением по грузоподъемности [Электронный ресурс] // Труды Института системного программирования РАН. — 2018. — URL: <https://ispranproceedings.elpub.ru/jour/article/view/1184> (дата обращения: 26.04.2026).

42. Левкин Г. Г., Куликова О. М. Аспекты управления логистикой на этапе последней мили [Электронный ресурс] // Вестник евразийской науки. — 2021. — URL: <https://cyberleninka.ru/article/n/aspekty-upravleniya-logistik>

оу-na-etape-posledney-mili (дата обращения: 26.04.2026).

43. Майерс Г., Сандлер К., Баджетт Т. Искусство тестирования программ. — 3-е изд. — Москва: Диалектика, 2018. — 272 с. — ISBN 978-5-8459-1796-6.

44. API Яндекс Карты: документация для разработчиков [Электронный ресурс]. — Режим доступа: <https://yandex.ru/dev/maps/>, свободный (дата обращения: 24.04.2026).

45. 2ГИС: документация для разработчиков [Электронный ресурс]. — Режим доступа: <https://docs.2gis.com/ru/>, свободный (дата обращения: 24.04.2026).

46. The Linux Kernel Archives [Электронный ресурс] / The Linux Kernel Organization. — Режим доступа: <https://www.kernel.org/>, свободный (дата обращения: 07.05.2026).

47. macOS [Электронный ресурс] / Apple Inc. — Режим доступа: <https://www.apple.com/macos/>, свободный (дата обращения: 07.05.2026).

48. Windows 11: Features and Benefits [Электронный ресурс] / Microsoft Corporation. — Режим доступа: <https://www.microsoft.com/en-us/windows/windows-11>, свободный (дата обращения: 07.05.2026).

49. Tailwind CSS: Responsive design [Электронный ресурс]. — Режим доступа: <https://tailwindcss.com/docs/responsive-design>, свободный (дата обращения: 07.05.2026).

50. Holt M. Papa Parse: The powerful, in-browser CSV parser for big boys and girls [Электронный ресурс]. — Режим доступа: <https://www.papaparse.com/docs>, свободный (дата обращения: 07.05.2026).

51. SheetJS Community Edition: Spreadsheet Data Toolkit [Электронный ресурс] / SheetJS LLC. — Режим доступа: <https://docs.sheetjs.com/>, свободный (дата обращения: 07.05.2026).

52. shadcn/ui: A set of beautifully-designed, accessible components and a code distribution platform [Электронный ресурс]. — Режим доступа: <https://ui>

.shadcn.com/docs, свободный (дата обращения: 07.05.2026).

53. Radix Primitives: Unstyled, accessible, open source React primitives for high-quality web apps and design systems [Электронный ресурс]. — Режим доступа: <https://www.radix-ui.com/primitives>, свободный (дата обращения: 07.05.2026).

54. Node.js Documentation [Электронный ресурс] / OpenJS Foundation. — Режим доступа: <https://nodejs.org/docs/>, свободный (дата обращения: 07.05.2026).

55. npm: Fast, disk space efficient package manager [Электронный ресурс]. — Режим доступа: <https://npm.io/>, свободный (дата обращения: 07.05.2026).

56. Mock Service Worker: API mocking library for browser and Node.js [Электронный ресурс]. — Режим доступа: <https://mswjs.io/docs/>, свободный (дата обращения: 07.05.2026).

57. golang-migrate/migrate: Database migrations written in Go [Электронный ресурс]. — Режим доступа: <https://github.com/golang-migrate/migrate>, свободный (дата обращения: 07.05.2026).

58. stretchr/testify: A toolkit with common assertions and mocks that plays nicely with the standard library [Электронный ресурс]. — Режим доступа: <https://github.com/stretchr/testify>, свободный (дата обращения: 07.05.2026).

59. Vitest: Next Generation testing framework powered by Vite [Электронный ресурс]. — Режим доступа: <https://vitest.dev/guide/>, свободный (дата обращения: 07.05.2026).

60. React Testing Library: Simple and complete testing utilities that encourage good testing practices [Электронный ресурс]. — Режим доступа: <https://testing-library.com/docs/react-testing-library/intro/>, свободный (дата обращения: 07.05.2026).

61. Secure Hash Standard (SHS): Federal Information Processing Standards Publication FIPS PUB 180-4 / National Institute of Standards

and Technology. — Gaithersburg: U.S. Department of Commerce, 2015. — DOI: 10.6028/NIST.FIPS.180-4. — Режим доступа: <https://doi.org/10.6028/NIST.FIPS.180-4>, свободный (дата обращения: 07.05.2026).

ПРИЛОЖЕНИЕ А

(справочное)

Схема базы данных и конфигурация контейнеризации

Схема базы данных (SQL-миграции)

Листинг А.1 – Миграция 0001_init.up.sql — инициализация схемы базы данных

```
1 CREATE EXTENSION IF NOT EXISTS "uuid-oss";

2 -- users
3 CREATE TABLE IF NOT EXISTS users (
4     id          UUID          PRIMARY KEY DEFAULT
    ↪  uuid_generate_v4(),
5     email       VARCHAR(255) NOT NULL UNIQUE,
6     password_hash VARCHAR(255) NOT NULL,
7     created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
8     updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
9 );

10 -- refresh_tokens
11 CREATE TABLE IF NOT EXISTS refresh_tokens (
12     id          UUID          PRIMARY KEY DEFAULT
    ↪  uuid_generate_v4(),
13     user_id     UUID          NOT NULL REFERENCES users(id) ON
    ↪  DELETE CASCADE,
14     token_hash  VARCHAR(255) NOT NULL UNIQUE,
15     expires_at  TIMESTAMPTZ NOT NULL,
16     revoked_at  TIMESTAMPTZ NULL,
17     created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
18 );
19 CREATE INDEX IF NOT EXISTS refresh_tokens_user_id_idx
```

```

20     ON refresh_tokens(user_id);

21 -- tasks
22 CREATE TABLE IF NOT EXISTS tasks (
23     id          UUID          PRIMARY KEY DEFAULT
    ↪ uuid_generate_v4(),
24     user_id     UUID          NOT NULL REFERENCES users(id)
    ↪ ON DELETE CASCADE,
25     title       VARCHAR(200) NOT NULL,
26     address_text VARCHAR(400) NOT NULL,
27     latitude    NUMERIC(9,6) NULL,
28     longitude   NUMERIC(9,6) NULL,
29     duration_min INT          NOT NULL,
30     window_start TIME         NULL,
31     window_end  TIME         NULL,
32     sort_index  INT          NOT NULL DEFAULT 0,
33     created_at  TIMESTAMPTZ  NOT NULL DEFAULT now(),
34     updated_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
35 );

36 CREATE INDEX IF NOT EXISTS tasks_user_id_idx
37     ON tasks(user_id);
38 CREATE INDEX IF NOT EXISTS tasks_user_sort_idx
39     ON tasks(user_id, sort_index);

40 -- routes (один маршрут на пользователя)
41 CREATE TABLE IF NOT EXISTS routes (
42     id          UUID          PRIMARY KEY DEFAULT
    ↪ uuid_generate_v4(),
43     user_id     UUID          NOT NULL REFERENCES users(id) ON
    ↪ DELETE CASCADE,
44     status      VARCHAR(30) NOT NULL,      -- optimized
45     source      VARCHAR(30) NOT NULL,      -- optimized
46     algorithm   VARCHAR(50) NULL,
47     started_at  TIMESTAMPTZ NULL,

```

```

48     finished_at TIMESTAMPTZ NULL,
49     created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
50     UNIQUE (user_id)
51 );

52 -- route_stats (1:1 to routes)
53 CREATE TABLE IF NOT EXISTS route_stats (
54     route_id          UUID PRIMARY KEY REFERENCES routes(id)
55     ↪ ON DELETE CASCADE,
56     total_distance_m  INT NULL,
57     total_travel_sec  INT NULL,
58     total_service_sec INT NULL,
59     total_wait_sec    INT NULL,
60     computed_at       TIMESTAMPTZ NOT NULL DEFAULT now()
61 );

62 -- route_stops
63 CREATE TABLE IF NOT EXISTS route_stops (
64     id                UUID PRIMARY KEY DEFAULT
65     ↪ uuid_generate_v4(),
66     route_id          UUID NOT NULL REFERENCES routes(id)
67     ↪ ON DELETE CASCADE,
68     task_id           UUID NOT NULL REFERENCES tasks(id)
69     ↪ ON DELETE CASCADE,
70     position          INT NOT NULL,
71     travel_from_prev_sec INT NULL,
72     arrive_time        TIMESTAMPTZ NULL,
73     service_start_time TIMESTAMPTZ NULL,
74     service_end_time   TIMESTAMPTZ NULL,
75     wait_sec           INT NULL
76 );

77 CREATE INDEX IF NOT EXISTS route_stops_route_id_idx
78     ON route_stops(route_id);

```

```

75 CREATE UNIQUE INDEX IF NOT EXISTS
    ↪ route_stops_route_position_uq
76 ON route_stops(route_id, position);

77 -- route_geometry (1:1 к routes)
78 CREATE TABLE IF NOT EXISTS route_geometry (
79     route_id    UUID PRIMARY KEY REFERENCES routes(id) ON
    ↪ DELETE CASCADE,
80     polyline    TEXT NOT NULL,
81     bbox_json   JSONB NULL,
82     geojson_json JSONB NULL,
83     updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()
84 );

```

Листинг А.2 – Миграция 0002_routes_name.up.sql — добавление поля name

```

1 ALTER TABLE routes ADD COLUMN IF NOT EXISTS name
    ↪ VARCHAR(200) NULL;

```

Листинг А.3 – Миграция 0003_task_completed.up.sql — добавление признака выполнения

```

1 ALTER TABLE tasks ADD COLUMN IF NOT EXISTS is_completed
    ↪ BOOLEAN NOT NULL DEFAULT false;

```

Листинг А.4 – Миграция 0004_task_duration_optional.up.sql — опциональная длительность

```

1 ALTER TABLE tasks ALTER COLUMN duration_min DROP NOT NULL;

```

Листинг А.5 – Миграция 0005_task_datetime_windows.up.sql — отдельные дата и время окна

```

1 -- Разделяем window_start/window_end (TIME) на отдельные
   ↳ колонки дата + время
2 ALTER TABLE tasks
3     DROP COLUMN window_start,
4     DROP COLUMN window_end;

5 ALTER TABLE tasks
6     ADD COLUMN window_start_date DATE NULL,
7     ADD COLUMN window_start_time TIME NULL,
8     ADD COLUMN window_end_date DATE NULL,
9     ADD COLUMN window_end_time TIME NULL;

```

Конфигурация контейнеризации

Листинг А.6 – Конфигурация docker-compose.yml серверной части проекта

```

1 services:
2   db:
3     image: postgres:16
4     environment:
5       POSTGRES_DB: ${POSTGRES_DB}
6       POSTGRES_USER: ${POSTGRES_USER}
7       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
8     ports:
9       - "5432:5432"
10    volumes:
11      - db_data:/var/lib/postgresql/data
12    healthcheck:
13      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d
   ↳ ${POSTGRES_DB}"]
14      interval: 5s

```

```

15     timeout: 3s
16     retries: 20

17 migrate:
18     image: migrate/migrate:v4.17.1
19     depends_on:
20         db:
21             condition: service_healthy
22     volumes:
23         - ./migrations:/migrations
24     entrypoint:
25         - "migrate"
26         - "-path"
27         - "/migrations"
28         - "-database"
29         - "postgres://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db_
↵ :5432/${POSTGRES_DB}?sslmode=disable"
30         - "up"

31 api:
32     build: .
33     depends_on:
34         db:
35             condition: service_healthy
36     migrate:
37         condition: service_completed_successfully
38     environment:
39         APP_HTTP_ADDR: "${APP_HTTP_ADDR}"
40         APP_DB_DSN: "postgres://${POSTGRES_USER}:${POSTGRES_PA
↵ SWORD}@db:5432/${POSTGRES_DB}?sslmode=disable"
41         APP_JWT_SECRET: "${APP_JWT_SECRET}"
42         APP_ACCESS_TTL_MIN: "${APP_ACCESS_TTL_MIN}"
43         APP_REFRESH_TTL_DAYS: "${APP_REFRESH_TTL_DAYS}"
44         APP_CORS_ORIGINS: "${APP_CORS_ORIGINS}"

```

```
45     APP_OSRM_BASE_URL: "${APP_OSRM_BASE_URL}"
46     ports:
47         - "8080:8080"

48 volumes:
49     db_data:
```


ПРИЛОЖЕНИЕ Б

(справочное)

Листинги ключевых фрагментов кода

Листинг Б.1 – Алгоритм ближайшего соседа с временными окнами и просмотром вперед (nearest_neighbor.go)

```
1 package optimizer

2 import (
3     "context"
4     "math"
5 )

6 // NearestNeighborTW – эвристика ближайшего соседа с учетом
   ↳ временных окон
7 // (NNH-TW). Все временные величины хранятся в секундах
   ↳ Unix. Сложность –  $O(n^3)$ .
8 type NearestNeighborTW struct{}

9 func NewNearestNeighborTW() *NearestNeighborTW { return
   ↳ &NearestNeighborTW{} }
10 func (a *NearestNeighborTW) Name() string { return
   ↳ "nearest-neighbor-tw" }

11 // traversal инкапсулирует состояние обхода: текущая
   ↳ позиция, тайминги,
12 // агрегированные показатели; appendNode инкрементально его
   ↳ обновляет.
13 type traversal struct {
14     g
   ↳ *Graph
15     cur
   ↳ int
```

```

16     currentTime
    ↪     int64
17     visited
    ↪     []bool
18     order
    ↪     []int
19     timings
    ↪     []StopTiming
20     totalDistM, totalTravelSec, totalServiceSec,
    ↪     totalWaitSec int
21 }

22 func (t *traversal) appendNode(next int) {
23     e := t.g.Edges[t.cur][next]
24     node := t.g.Nodes[next]
25     arrivalSec := t.currentTime + int64(e.DurationSec)
26     var waitSec int64
27     if node.WindowStart >= 0 && arrivalSec <
    ↪     node.WindowStart {
28         waitSec = node.WindowStart - arrivalSec
29     }
30     serviceStart := arrivalSec + waitSec
31     serviceEnd := serviceStart + int64(node.DurationMin)*60

32     t.totalDistM += e.DistanceM
33     t.totalTravelSec += e.DurationSec
34     t.totalServiceSec += node.DurationMin * 60
35     t.totalWaitSec += int(waitSec)

36     t.timings = append(t.timings, StopTiming{
37         NodeIdx: next, ArrivalSec: arrivalSec, WaitSec:
    ↪     int(waitSec),
38         ServiceStartSec: serviceStart, ServiceEndSec:
    ↪     serviceEnd,

```

```

39         TravelFromPrevSec: e.DurationSec,
40     })
41     t.visited[next] = true
42     t.order = append(t.order, next)
43     t.cur = next
44     t.currentTime = serviceEnd
45 }

46 func (a *NearestNeighborTW) Optimize(
47     _ context.Context, g *Graph, startTimeUnix int64, c
48     ↪ Constraints,
49 ) (Result, error) {
50     n := len(g.Nodes)
51     if n == 0 {
52         return Result{}, nil
53     }
54     prereqs := buildPrereqs(n, c.PrecedencePairs)
55     endIdx := -1
56     if c.EndNodeIdx != nil {
57         endIdx = *c.EndNodeIdx
58     }
59
60     cur := 0
61     if c.StartNodeIdx != nil {
62         cur = *c.StartNodeIdx
63     } else {
64         cur = startNode(g, prereqs, endIdx)
65     }
66     startServiceEnd := startTimeUnix +
67     ↪ int64(g.Nodes[cur].DurationMin)*60
68     t := &traversal{
69         g: g, cur: cur, currentTime: startServiceEnd,
70         visited: make([]bool, n), order: []int{cur},
71         timings: []StopTiming{{

```

```

69         NodeIdx: cur, ArrivalSec: startTimeUnix,
70         ServiceStartSec: startTimeUnix, ServiceEndSec:
↪ startServiceEnd,
71     }},
72     totalServiceSec: g.Nodes[cur].DurationMin * 60,
73 }
74 t.visited[cur] = true

75 for len(t.order) < n {
76     // Если остался только закреплённый конечный узел —
↪ ставим его и выходим.
77     if endIdx >= 0 && !t.visited[endIdx] &&
↪ onlyEndUnvisited(t.visited, endIdx) {
78         t.appendNode(endIdx)
79         break
80     }
81     next := pickNextFeasible(t, prereqs, endIdx)
82     if next == -1 {
83         next = pickNextFallback(t, prereqs, endIdx)
84     }
85     if next == -1 {
86         // Циклический граф предшествования — конечный
↪ узел добавим и завершим.
87         if endIdx >= 0 && !t.visited[endIdx] {
88             t.appendNode(endIdx)
89         }
90         break
91     }
92     t.appendNode(next)
93 }
94 return Result{
95     Order: t.order, Timings: t.timings,
96     TotalDistanceM: t.totalDistM, TotalTravelSec:
↪ t.totalTravelSec,

```

```

97         TotalServiceSec: t.totalServiceSec, TotalWaitSec:
↪     t.totalWaitSec,
98     }, nil
99 }

100 // pickNextFeasible – проход 1: допустимый узел с
↪     минимальным временем
101 // завершения + look-ahead. Безопасные кандидаты
↪     предпочитают.
102 func pickNextFeasible(t *traversal, prereqs [][]int, endIdx
↪     int) int {
103     g := t.g
104     next := -1
105     var bestComp int64 = math.MaxInt64
106     var bestDeadline int64 = math.MaxInt64
107     bestSafe := false
108     for i := range g.Nodes {
109         if t.visited[i] || i == endIdx || !prereqsMet(i,
↪     t.visited, prereqs) {
110             continue
111         }
112         arrival := t.currentTime +
↪     int64(g.Edges[t.cur][i].DurationSec)
113         if !feasible(g.Nodes[i], arrival) {
114             continue
115         }
116         comp := nodeCompletionTime(g.Nodes[i], arrival)
117         deadline := g.Nodes[i].WindowEnd
118         if deadline < 0 {
119             deadline = math.MaxInt64
120         }
121         safe := !causesWindowMiss(i, comp, g, t.visited,
↪     endIdx)
122         better := false

```

```

123         switch {
124         case safe && !bestSafe:
125             better = true
126         case safe == bestSafe:
127             better = comp < bestComp ||
128                 (comp == bestComp && deadline <
↪ bestDeadline)
129         }
130         if better {
131             bestComp = comp; bestDeadline = deadline
132             bestSafe = safe; next = i
133         }
134     }
135     return next
136 }

137 // pickNextFallback – проход 2: минимальный completionTime,
↪   окно игнорируется,
138 // предшествование сохраняется.
139 func pickNextFallback(t *traversal, prereqs [][]int, endIdx
↪   int) int {
140     g := t.g
141     next := -1
142     var bestComp int64 = math.MaxInt64
143     for i := range g.Nodes {
144         if t.visited[i] || i == endIdx || !prereqsMet(i,
↪ t.visited, prereqs) {
145             continue
146         }
147         arrival := t.currentTime +
↪ int64(g.Edges[t.cur][i].DurationSec)
148         comp := nodeCompletionTime(g.Nodes[i], arrival)
149         if comp < bestComp {
150             bestComp = comp; next = i

```

```

151         }
152     }
153     return next
154 }

155 func feasible(node Node, arrivalSec int64) bool {
156     if node.WindowEnd < 0 {
157         return true
158     }
159     return arrivalSec <=
    ↪ node.WindowEnd-int64(node.DurationMin)*60
160 }

161 // causesWindowMiss – look-ahead на один шаг: true, если
    ↪ после cand
162 // хотя бы один узел с окном становится недостижимым.
163 func causesWindowMiss(cand int, afterSec int64,
164     g *Graph, visited []bool, endIdx int) bool {
165     for j := 0; j < len(g.Nodes); j++ {
166         if visited[j] || j == cand || j == endIdx {
167             continue
168         }
169         if g.Nodes[j].WindowEnd < 0 {
170             continue
171         }
172         arr := afterSec +
    ↪ int64(g.Edges[cand][j].DurationSec)
173         if !feasible(g.Nodes[j], arr) {
174             return true
175         }
176     }
177     return false
178 }

```

Листинг Б.2 – HTTP-обработчик оптимизации маршрута
(handlers_routes.go, метод Optimize)

```
1 // Optimize - POST /api/routes/optimize.
2 func (h *RouteHandlers) Optimize(c *gin.Context) {
3     userID, ok := userIDFromCtx(c)
4     if !ok {
5         return
6     }
7
8     var req OptimizeRouteReq
9     if err := c.ShouldBindJSON(&req); err != nil {
10         c.JSON(http.StatusBadRequest, gin.H{"error": "bad
↵ request"})
11         return
12     }
13
14     startTime := req.StartTimeUnix
15     if startTime <= 0 {
16         startTime = time.Now().Unix()
17     }
18     startTime = timing.RoundUpUnixToMinute(startTime)
19
20     matrix := convertMatrix(req.DistanceMatrix)
21     precedences :=
↵ convertPrecedences(req.PrecedenceConstraints)
22
23     out, err := h.story.Optimize(c.Request.Context(),
↵ userID,
24         req.TaskIDs, startTime, matrix,
25         req.StartTaskID, req.EndTaskID, precedences)
26     if err != nil {
27         c.JSON(http.StatusBadRequest, gin.H{"error":
↵ err.Error()})
28     }
```



```

24         return
25     }
26     c.JSON(http.StatusOK, out)
27 }

28 func convertMatrix(in [][]DistanceCellDTO) [][]distance.Edge
    ↪ {
29     if len(in) == 0 {
30         return nil
31     }
32     out := make([][]distance.Edge, len(in))
33     for i, row := range in {
34         out[i] = make([]distance.Edge, len(row))
35         for j, cell := range row {
36             out[i][j] = distance.Edge{
37                 DistanceM:    cell.DistanceM,
38                 DurationSec:
    ↪ timing.RoundUpSecToMinute(cell.DurationSec),
39             }
40         }
41     }
42     return out
43 }

44 func convertPrecedences(in []PrecedencePairDTO)
    ↪ []routestory.PrecedenceConstraint {
45     out := make([]routestory.PrecedenceConstraint, len(in))
46     for i, p := range in {
47         out[i] = routestory.PrecedenceConstraint{
48             BeforeTaskID: p.BeforeTaskID,
49             AfterTaskID:   p.AfterTaskID,
50         }
51     }
52     return out

```

```

53 }

54 // OptimizeRouteReq – тело POST /api/routes/optimize.
55 // Если DistanceMatrix передана – бэкенд не ходит в OSRM
56 // и порядок согласован с Yandex.
57 type OptimizeRouteReq struct {
58     TaskIDs []string
59     ↪ `json:"taskIds"`
60     StartTimeUnix int64
61     ↪ `json:"startTimeUnix"`
62     DistanceMatrix [][]DistanceCellDTO
63     ↪ `json:"distanceMatrix,omitempty"`
64     StartTaskID *string
65     ↪ `json:"startTaskId,omitempty"`
66     EndTaskID *string
67     ↪ `json:"endTaskId,omitempty"`
68     PrecedenceConstraints []PrecedencePairDTO
69     ↪ `json:"precedenceConstraints,omitempty"`
70 }

```

Листинг Б.3 – Ключевые функции API-клиента: оптимизация маршрута и обновление токена (client.ts)

```

1 let refreshSessionPromise: Promise<AuthSession> | null =
  ↪ null;

2 export interface PrecedenceConstraint {
3     beforeTaskId: string;
4     afterTaskId: string;
5 }

6 // optimizeRoute передает идентификаторы задач, матрицу
  ↪ расстояний

```

```

7 // и дополнительные ограничения на бэкенд.
8 export async function optimizeRoute(
9   taskIds: string[],
10  options: AuthorizedRequestOptions,
11  startTimeUnix = 0,
12  distanceMatrix?: DistanceCell[][],
13  startTaskId?: string,
14  endTaskId?: string,
15  precedenceConstraints?: PrecedenceConstraint[],
16 ): Promise<SavedRouteDetails> {
17   const route = await requestWithAuth<ApiRouteFull>(
18     '/routes/optimize',
19     {
20       method: 'POST',
21       body: JSON.stringify({
22         taskIds, startTimeUnix, distanceMatrix,
23         startTaskId: startTaskId ?? undefined,
24         endTaskId: endTaskId ?? undefined,
25         precedenceConstraints: precedenceConstraints?.length
26           ? precedenceConstraints : undefined,
27       }),
28     },
29     options,
30   );
31   return mapRouteDetails(route);
32 }

33 // requestWithAuth выполняет HTTP-запрос с токеном доступа.
34 // При получении 401 однократно обновляет токен и повторяет
35   ↪ запрос.
36 async function requestWithAuth<T>(
37   path: string, init: RequestInit,
38   options: AuthorizedRequestOptions,
39 ): Promise<T> {

```

```

39   try {
40     return await request<T>(path, init,
↪    options.session.accessToken);
41   } catch (error) {
42     if (!(error instanceof ApiError) || error.status !==
↪    401) throw error;
43   }
44   try {
45     const nextSession = await refreshSession(options);
46     return request<T>(path, init, nextSession.accessToken);
47   } catch (error) {
48     await options.onUnauthorized();
49     throw error;
50   }
51 }

52 // refreshSession — дедупликация через единственный промис.
53 async function refreshSession(options:
↪   AuthorizedRequestOptions) {
54   if (!refreshSessionPromise) {
55     refreshSessionPromise =
↪     request<TokenPairResponse>('/auth/refresh', {
56       method: 'POST',
57       body: JSON.stringify({
58         refreshToken: options.session.refreshToken,
59       }),
60     })
61     .then((pair) => {
62       const nextSession = buildSession(pair,
63         getEmailFromToken(pair.accessToken) ??
↪     options.session.email);
64       options.onSessionChange(nextSession);
65       return nextSession;
66     })

```

```

67         .finally(() => { refreshSessionPromise = null; });
68     }
69     return refreshSessionPromise;
70 }

```

Листинг Б.4 – Построение матрицы расстояний через Яндекс Маршруты JS API (routeOptimizer.ts)

```

1 import { Task } from '../types/task';
2 import { loadYandexMaps } from './yandexMaps';
3 import { roundUpSecToMinute } from './time';

4 export interface DistanceCell {
5     distanceM: number;
6     durationSec: number;
7 }

8 /**
9  * Строит матрицу расстояний n×n для списка задач через
   ↪ Яндекс Маршруты.
10  * Используется тот же движок, что и для визуализации
   ↪ маршрута на карте,
11  * поэтому данные оптимизации согласованы с отображением.
12  *
13  * matrix[i][j] = стоимость проезда от tasks[i] до tasks[j].
14  * Диагональные элементы (i === j) равны нулю.
15  */
16 export async function buildYandexDistanceMatrix(
17     tasks: Task[],
18     routingMode: 'auto' | 'pedestrian' | 'masstransit' =
   ↪ 'auto',
19 ): Promise<DistanceCell[][]> {
20     const n = tasks.length;

```

```

21   if (n === 0) return [];

22   await loadYandexMaps();

23   // masstransit через multiRouter не возвращает надежные
↪   данные по времени —
24   // используем 'auto' как приближение.
25   const mode = routingMode === 'masstransit' ? 'auto' :
↪   routingMode;

26   const fallbackSec = 99_999;
27   const matrix: DistanceCell[][] = Array.from({ length: n
↪   }, (_, i) =>
28     Array.from({ length: n }, (_, j) =>
29       i === j
30       ? { distanceM: 0, durationSec: 0 }
31       : { distanceM: 0, durationSec: fallbackSec },
32     ),
33   );

34   // Запрашиваем все пары параллельно: n*(n-1) запросов.
35   const pairs: [number, number][] = [];
36   for (let i = 0; i < n; i++)
37     for (let j = 0; j < n; j++)
38       if (i !== j) pairs.push([i, j]);

39   await Promise.all(
40     pairs.map(async ([i, j]) => {
41       const from = tasks[i];
42       const to = tasks[j];
43       if (from.latitude == null || to.latitude == null)
↪     return;

44       try {

```

```

45         const { distanceM, durationSec } = await new
↪ Promise<{
46             distanceM: number;
47             durationSec: number;
48         }>((resolve, reject) => {
49             const mr = new (window.ymaps as
↪ any).multiRouter.MultiRoute(
50                 {
51                     referencePoints: [
52                         [from.latitude!, from.longitude!],
53                         [to.latitude!, to.longitude!],
54                     ],
55                     params: { routingMode: mode },
56                 },
57                 {},
58             );

59         mr.model.events.once('requestsuccess', () => {
60             try {
61                 const activeRoute: any = mr.getActiveRoute();
62                 if (activeRoute) {
63                     const distObj: any =
↪ activeRoute.properties.get('distance');
64                     const durObj: any =
↪ activeRoute.properties.get('duration');
65                     resolve({
66                         distanceM: Math.round(
67                             typeof distObj?.value === 'number' ?
↪ distObj.value : 0,
68                         ),
69                         durationSec: roundUpSecToMinute(
70                             typeof durObj?.value === 'number'
71                             ? durObj.value
72                             : fallbackSec,

```

```

73         ),
74     });
75     return;
76 }

77     // Запасной путь: данные из модельных
↪ маршрутов напрямую.
78     const modelRoutes: any[] =
↪ mr.model.getRoutes();
79     if (modelRoutes.length > 0) {
80         const legs: any[] =
↪ modelRoutes[0].getLegs();
81         let totalDistM = 0;
82         let totalDurSec = 0;
83         legs.forEach((leg: any) => {
84             const d: any =
↪ leg.properties?.get?.('distance');
85             const dur: any =
↪ leg.properties?.get?.('duration');
86             totalDistM += typeof d?.value === 'number'
↪ ? d.value : 0;
87             totalDurSec +=
88                 typeof dur?.value === 'number' ?
↪ dur.value : 0;
89         });
90         resolve({
91             distanceM: Math.round(totalDistM),
92             durationSec:
93                 totalDurSec > 0
94                 ? roundUpSecToMinute(totalDurSec)
95                 : fallbackSec,
96         });
97         return;
98     }

```



```

99         reject(new Error('no route data'));
100     } catch (e) {
101         reject(e);
102     }
103     });

104     mr.model.events.once('requestfail', () => {
105         reject(new Error('routing failed'));
106     });
107 });

108     matrix[i][j] = { distanceM, durationSec };
109     } catch {
110         // Оставляем fallback: алгоритм избегает
↪     недостижимых пар.
111     }
112     },
113 );

114     return matrix;
115 }

```